南京大學

研 究 生 毕 业 论 文
（申请工程硕士学位）

论 文 题 目 基于算法模板推理的大语言模型的代码生成技术研究

作 者 姓 名 张浩天

学科、专业名称 软件工程

研 究 方 向 工程硕士（软件工程领域）

指 导 教 师 葛季栋 副教授

2024 年 **8** 月 **26** 日

学　　号：**522022320200**

论文答辩日期：**2024** 年 **8** 月 **27** 日

指 导 教 师：　　（签字）

Research on the Technology of Algorithm Template Inference for Large Language Model based Code Generation

by

ZHANG Hao-Tian

Supervised by

Associate Professor GE Ji-Dong

A dissertation submitted to
the graduate school of Nanjing University
in partial fulfilment of the requirements for the degree of
MASTER OF ENGINEERING
in
Software Engineering

Software Institute
Nanjing University

August 16,2024

南京大学研究生毕业论文中文摘要首页用纸

毕业论文题目：基于算法模板推理的大语言模型的代码生成技术研究

软件工程 专业 2022 级硕士生姓名： 张浩天

指导教师（姓名、职称）： 葛季栋 副教授

摘 要

随着近年来大语言模型应用的“爆炸式”发展，大型语言模型已经展示了其令人惊叹的代码生成能力，但最近关于这种能力是否是基于其潜在的数据污染的的争议却也喧嚣尘上。本文的研究旨在探讨如何增强大语言模型的代码生成能力，我们的研究发现，代码生成任务是衡量大语言模型推理能力的一个重要基准。尤其是在竞赛级编程题目的解决上，这些题目往往是各大互联网企业招聘员工时必不可少的测试环节，也是考查一名软件开发从业人员的编码能力和思维能力的主要手段。然而，即使最新的大语言模型在与人沟通和逻辑推理上表现出了极高的逻辑性和正确率，但其解决代码生成相关问题，尤其是竞赛级编程题上的能力依然饱受争议。许多相关研究都证明了大语言模型在复杂编程问题的解决上依然存在弱点，这些由大语言模型生成的代码很多时候都无法通过第一个测试用例。这说明主要的问题可能会集中在对于题目的理解上。

根据上述研究动机，致力于研究一种新的增强大语言模型代码生成能力的方法，本文提出了一种基于算法模板推理 ATI（Algorithm Template Inference）的方法来增强大语言模型的代码生成能力。ATI 方法首先通过问题解析器来提取题目中的关键信息和解题限制，确保大语言模型可以充分理解题目要求。然后利用问题解析器提取出来的问题信息来检索相关的算法模板，最后基于题目关键信息、解题限制和算法模板来进行更有效的题目推理。同时对于算法模板库，我们会根据解题过程对其进行扩展，使大模型的稳定性和可扩展性得到进一步的提升。我们选取了在大语言模型代码生成领域取得了优异成绩的四种推理增强方法，包括 CoT、PAL、ToT 和 GoT。本文通过在多语言代码生成数据集 Humaneval-X 和我们自研的竞赛级代码题数据集 TemplateLeetcode 上进行实验，并对比了 ATI 与其他四种方法的性能差异，本文提出的 ATI 方法能在推理速度和推理准确性上超越已有的所有方法。和综合最优方法 ToT 相比，ATI

方法在消耗 ToT 方法 60% 的检索时间下，平均 pass@ k 值比 ToT 方法提高了 8%。

关键词： 代码生成；检索增强生成；深度学习；思维链

南京大学研究生毕业论文英文摘要首页用纸

THESIS: Research on the Technology of Algorithm Template Inference for Large Language Model based Code Generation

SPECIALIZATION: Software Engineering

POSTGRADUATE: ZHANG Hao-Tian

MENTOR: Associate Professor GE Ji-Dong

Abstract

With the ”explosive” growth in the application of large language models in recent years, these models have demonstrated impressive code generation capabilities. However, there has been a rising controversy over whether these abilities are a result of potential data contamination. This study aims to explore how to enhance the code generation capabilities of large language models. Our research has found that code generation tasks are a crucial benchmark for evaluating the reasoning abilities of large language models. This is especially true in solving competition-level programming problems, which are often essential tests in the recruitment process of major internet companies and a key method for assessing a software developer’s coding and problem-solving skills.However, even the latest large language models, which exhibit high logical consistency and accuracy in communication and reasoning, still face significant challenges in code generation, particularly in solving competition-level programming problems. Numerous studies have shown that large language models still have weaknesses in solving complex programming problems, with the generated code often failing to pass the first test case. This suggests that the main issue may lie in the model’s understanding of the problem.

Based on the above research motivation and aiming to develop a new method to enhance the code generation capabilities of large language models, this paper proposes an Algorithm Template Inference (ATI) method to improve these models’ code generation capabilities. The ATI method first utilizes a problem parser to extract key information and constraints from the problem to ensure that the large language model fully understands the problem requirements. It then retrieves relevant algorithm tem-

plates based on the extracted information. Finally, it performs more effective problem reasoning by combining key problem information, constraints, and algorithm templates.Furthermore, the algorithm template library will be expanded according to the problem-solving process, further improving the model's stability and scalability. We selected four inference enhancement methods that have achieved excellent results in the field of large language model code generation, including CoT, PAL, ToT, and GoT. Experiments were conducted on the multilingual code generation dataset Humaneval-X and our self-developed competition-level code problem dataset TemplateLeetcode. The ATI method proposed in this paper outperformed all existing methods in both inference speed and accuracy. Compared to the comprehensive optimal benchmark method ToT, the ATI method increases the average pass@ k value by 8% while consuming only 60% of ToT's retrieval time.

keywords: Code Generation, Retrieval Augmented Generation, Deep Learning, Chain of Thought

目　　录

目　　录 ····· v
插图清单 ····· vii
附表清单 ····· ix
第一章　引言 ····· 1
1.1　研究背景与意义 ····· 1
1.2　研究现状 ····· 2
1.2.1　基于思维链的大语言模型推理能力增强 ····· 2
1.2.2　基于预训练的大语言模型推理能力增强 ····· 5
1.3　本文贡献 ····· 8
1.4　本文组织结构 ····· 8
第二章　相关技术 ····· 11
2.1　检索增强生成技术 ····· 11
2.1.1　初级 RAG ····· 12
2.1.2　高级 RAG ····· 14
2.1.3　模块化 RAG ····· 14
2.2　文本嵌入模型技术 ····· 15
2.2.1　稀疏嵌入 ····· 16
2.2.2　密集嵌入 ····· 17
2.2.3　多向量 ····· 18
2.3　思维增强方法 ····· 20
2.4　Transformer 模型 ····· 23
2.4.1　编码器（Encoder） ····· 24
2.4.2　解码器（Decoder） ····· 24
2.4.3　自注意力机制（Self-Attention Mechanism） ····· 25
2.4.4　前馈神经网络（Feed-Forward Network） ····· 25
2.5　本章小结 ····· 27

第三章 算法模板库方法 ····· **29**
3.1 研究动机 ····· 29
3.2 算法模板库方法概述 ····· 32
3.3 问题解析阶段 ····· 33
3.4 算法模板匹配阶段 ····· 34
3.5 本章小结 ····· 38
第四章 实验设计与结果分析 ····· **39**
4.1 实验设计 ····· 39
4.1.1 基准方法 ····· 39
4.1.2 实验数据 ····· 40
4.1.3 评估指标和实现细节 ····· 41
4.2 实验结果分析 ····· 43
4.2.1 实验结果 ····· 43
4.3 消融实验 ····· 54
4.3.1 本章小结 ····· 55
第五章 总结 ····· **57**
5.1 工作总结 ····· 57
5.2 研究展望 ····· 58
致 谢 ····· **61**
参考文献 ····· **63**
简历与科研成果 ····· **73**
《学位论文出版授权书》 ····· **75**

插图清单

1-1 基于 SCoT 构建提示链的示例 ····· 4

2-1 RAG 架构 ····· 12
2-2 colbert 结构模型 ····· 19
2-3 CoT 结构模型 ····· 20
2-4 PAL 模型示例 ····· 21
2-5 ToT 和 CoT 的对比，每个矩形框代表一个推理想法 ····· 22
2-6 GoT 模型结构 ····· 22
2-7 Transformers 架构示意图 ····· 23

3-1 ATI 流程图 ····· 32
3-2 问题解析器示例 ····· 34
3-3 滑动窗口类型算法题模板 ····· 36

4-1 使用 CoT 方法的代码实现 ····· 44
4-2 使用 ToT 方法的代码实现 ····· 46
4-3 使用 ATI 方法的代码实现 ····· 50
4-4 基于 HumanEvalX 的不同提示链构造方法在三种大语言模型上指标的对比示意图 ····· 52
4-5 基于 TemplateLeetcode 的不同提示链构造方法在三种大语言模型上指标的对比示意图 ····· 53
4-6 在不同大语言模型上，我们的 ATI 方法与其他方法的对数推理时间比较 ····· 54

附表清单

3–1 GPT-4 在不同问题组上的表现 ····· 31

4–1 实验环境及工具版本 ····· 43
4–2 将 ATI 在各个模型上与之前各种增强代码生成方法进行比较 ····· 51
4–3 消融实验结果 ····· 55

第一章　引言

1.1　研究背景与意义

随着大语言模型在各个领域应用的逐步加深，给不同领域内的任务都提供了显著的性能改进，尤其是大语言模型的优秀的推理能力令人印象深刻。有越来越多的机构利用大语言模型强大的推理能力开发专注于代码生成方面的工具，来帮助软件开发人员更便捷更高效的完成工作。代码生成能力作为评估大语言模型推理能力的一个关键性指标，可以有效的反应大语言模型的推理强度和深度。然而，尽管这些专注于代码生成的大语言模型在很多时候都能够生成高质量的代码片段，它们仍然面临着一些挑战。Yiming Huang 等人 [1] 在论文中指出“即使是最新的大语言模型在代码生成方面的也具有内在局限性，这表现在大语言模型经常会误解用户的描述”，为了解决这一问题，最便捷的方法似乎就是通过提高模型的规模来进一步提高模型的推理能力，但这种便捷会带来成本的“爆炸”。因此寻找一种在不改变模型规模的基础上来提高代码生成准确性的方法，是迫在眉睫的，也是具有重大研究意义的。

如何写出高质量的可用代码，在特定的生产工作中既可以提高开发效率，又可以降低后期维护费用。这是每一个软件开发人员都要面对的难题。随着软件规模不断扩张，软件功能逐渐多元，软件也日趋国际化、开放化、泛用化，构成软件的代码的复杂度也呈现指数级增长。如今即使是实现一个单一的功能，也需要软件开发人员设计许多复杂的代码块来增强这个功能的效率和稳定性。这些代码块既包括实现软件功能的综合代码，也包括为了优化功能而设计的算法代码。一方面，这些为了优化系统而存在的算法代码反而对程序员提出了更高的要求，其造成的时间开销也经常高于编写综合代码。另一方面，越是成熟的先进的互联网公司对程序员编写算法代码能力的要求就越高。例如，世界上排名前列的互联网公司都会把算法编程题作为招聘软件开发人员的第一环；很多大型互联网公司也会定期安排算法能力考察，即使是那些入职很久的资深程序员也不能免除。最后，开发基于大语言模型的代码生成工具，正在显著降低开发者的工作负担，减少人为出错的概率，并加快项目的开发进度，改

善项目性能，降低后期维护费用。这对于提升软件的开发效率，提升代码的品质，意义不容小觑。对这一方向的研究不仅具有理论意义，还具有广泛的实际应用前景，必将给软件开发领域带来更多的创新和突破。

1.2 研究现状

1.2.1 基于思维链的大语言模型推理能力增强

“Chain of Thought”[2]（简称 CoT）是一种旨在增强大语言模型推理能力的方法，通过让模型在生成过程中一步一步地展示推理步骤，使模型在每一步都能参照以往的推理过程，从而更好地理解和解决复杂问题。这种方法主要用于提高模型在复杂推理任务中的表现，如数学计算、逻辑推理和多步推理任务。

在现有大语言模型的研究中，根据是否展示推理步骤，可以将模型推理分为 Chain of Thought（CoT）和 no Chain of Thought[2]（no CoT）两种。no Chain of Thought 方法通常输出的是模型经过一次性分析生成的答案，即只根据输入信息直接生成结果。这种方法能够在简单任务中表现出色，但在复杂推理任务中往往显得不足。Chain of Thought 方法将逐步展示推理过程作为自己的推理路线，能够实现较为精细的推理并生成高质量的答案。但同时，Chain of Thought 方法也需要研究人员构造更复杂的模型架构，并且在某些情况下会生成冗长和不必要详细的答案。针对 CoT 方法，目前主流的研究方向被分为两种：基于人工示例标注的 Few-shot CoT[3] 和无人工示例标注的 Zero-shot CoT[4]。

基于人工示例标注的 Few-shot CoT 方法源于一个具有普适性的思想，即“一个复杂的推理任务，其可以有多种推理路径（即解题思路），这些推理路径往往最终都能够得到正确的答案”[3]。但很多时候大语言模型在推理的过程中会陷入一些错误的思维逻辑，这也导致大语言模型可能会产出虽然严格符合输出格式，但却有明显内容错误的结果。Few-shot CoT 方法旨在尝试不只是约束大语言模型的输出格式，同时也约束大语言模型的推理路径，使大语言模型可以在某条合理的思路上发散思维。这也会导致基于人工示例标注的 Few-shot CoT 方法高度依赖于软件开发人员的个人经验和对于问题的理解程度，虽然简单的思维链和复杂的思维链往往会生成不一样的推理结果，但这并不表明复杂一定优于简单，如何设置一个合适规模的提示链也是 Few-shot CoT 所面临的一大挑战。为了规范思维链的生成，Jason Wei[2] 等人给出了设计思维链的几种

策略：

- 结合验证与细化：基于测试覆盖信息进行定位；
- 问题分解：问题分解对多步骤推理题的中间步骤进行多次分解，并分配更多的运算量，产生更多的 token，最后基于这些分解出来的结果解决问题；
- 利用外部知识：在某些情况下，为了促进知识密集型任务，可以结合外部知识对不确定的例子进行重新推理，从而减少分析过程中出现的事实错误；
- 投票排序：DIVERSE 等策略利用投票机制消除错误答案，然后对每个推理步骤进行独立的细粒度验证，从而提高推理的准确；
- 提高效率：通过优化模型结构和算法，减少不必要的计算量，提高推理的效率。

与 Few-shot CoT 不同，Zero-shot CoT 并不需要人为构造提示，只需要在用户输入中加入一个特定的指令，即可驱动大语言模型按照思维链的方式生成结果。当然，这种不需要人工构造思维链的方式，效果相对于 Few-shot CoT 而言往往会表现得更差。但同时，Zero-shot CoT 在陌生问题和复杂问题上却能表现出显著的推理水平提高。这表明 Zero-shot CoT 可以利用大语言模型的知识储备和以往经验来弥补软件开发人员在一些陌生和复杂问题上见识的不足。同时这也证明了大语言模型可以在推理过程中进行自我限制。当然，这种让大语言模型来构建提示链的方式具有不稳定性，并且其效果也依赖于模型自身的规模和微调任务数量。

Jia Li 团队 [5] 在传统 CoT 的基础上，针对代码生成的问题，提出了面向代码生成的结构化思维链 SCoT（Structured Chain-of-Thought），它要求大模型用程序结构（例如：顺序、分支和循环结构）来组织思维过程，从程序语言的角度引导大模型思考如何解决需求。该设计的思想源于 1966 年由 Bohm 和 Jacopini [6] 提出的：“使用顺序结构、分支结构和循环结构这三种基本结构的组合，可以解决任何不论简单还是复杂的算法问题”。在具体的编码实践中，大多数程序的源代码都具有较强的结构性。在直觉上，一条结构化的思维链将有利于推导结构化的源代码。基于此，研究者通过这三个基本结构来约束大模型产生思维链条。SCoT 的解题过程如图 1–1所示，该思维链条要求大模型思考如何从程序化语言的角度去解决问题，如何使用这三种基本的架构去精准的表达思维过程。实验结果表明：在大语言模型的代码生成过程中加入结构化的思维链，可以让大语言模型稳定的生成比之前更高正确率的代码。

```
def first_Repeated_Char(str):
"""
Write a python function to find the first repeated character in a given
string.
"""
Please understand the requirement and write a rough solving process. It
starts with a input-output structure. You
should use three basic structures to build the solving
Example
process, including sequences,branches,and loops. The
necessary details should be written in natural languages.
Input: str: a string
0utput: ch: a repeated character in str
1: for each character ch in str:
"
2:      if ch appears more than once in str:
3:              return ch
4: return None

(more examples..)
```

Example

```
def text__lowercase_underscore(text):
"""
Write a function to find sequences of lowercase letters joined with an
underscore.
"""
Please understand the requirement and write a rough solving
process. It starts with a input-output structure. You
should use three basic structures to build the solvingprocess, including
sequences, branches, and loops. Thenecessary details should be written
in natural languages.
```

New input

图 1-1: 基于 SCoT 构建提示链的示例

以上方法都是基于“Let’s think step by step”的单查询思想，虽然这些方法往往利用较少的样本就可以大幅提高正确率，但它在解决那些比思维链中给出的提示还要难的问题上却表现不佳。为了应对这些由易到难的推理挑战，Yao Fu 等人 [7] 提出了以多个大的语言模型查询为基准，引出不同的合理推理路径，从而将复杂的问题分解成一个较简单的子问题序列的多查询推理方法。Denny Zhouy 等人 [8] 开发的 Least-to-Most 就是一个典型的例子，该策略的关键思想是先做问题拆分，将复杂问题拆分为可以被快速解决的简单子问题，然后按顺序解决它们，并利用先前解决的问题的答案来促进其他子问题的解决。

基于这种模拟人类思维的自上而下的问题分解和自下而上的解决方案生成的方法，Least-to-Most 方法可以有效的解决组合泛华类问题。Shunyu Yao 等人 [9] 使用思维树（ToT）来提高大语言模型的前瞻能力和初始决策推理能力，通过考虑多种不同的推理路径和自我评估选择，以及在必要的情况下通过回溯做出全局选择来决定下一步的行动，进而提高大语言模型的前瞻性和最终的决策推理能力。Maciej Besta 等人 [10] 在提出的 Graph of Thoughts 方法中，将思维图引入到大语言模型的推理过程中，GoT 的关键思想和主要优势是能够将大语言模型生成的信息建模为任意图，其中推理思维是顶点，边对应于这些顶点之间的依赖关系。这种方法可以将任意的大语言模型思维组合产生协同效果，进而提炼整个思维网络的精髓或使用反馈循环来增强思维。这项工作使大语言模型的推理更接近人类的大脑机制。对比来看，GoT 可以弥补 ToT 方法通过将僵化的树结构强加于思维过程导致的从根本上限制了提示中的推理能力的问题。这说明人类在思考过程中不仅会遵循思维链（如 CoT）或尝试不同的独立思维（如 ToT），而实际上会形成更复杂的思维网络。例如，人们可以在探索某种推理链的同时回溯并开始新的推理链，然后意识到前一条链中的某个想法可以与当前探索的推理链相结合，并将其合并为一种可以利用前一个推理链的优势并消除其弱点的新的解决方案，这种复杂的思考其实就是一种类似于图的模式。

总的来说，思维链提示作为一种促进语言模型推理的方法具有很多优秀的特性。首先，思路链原则上允许模型将多步骤问题分解为中间步骤，这意味着其可以为需要更多推理步骤的问题分配额外的计算。其次，思路链为模型的推理行为提供了一个可解释的窗口，揭示了模型是如何得出特定答案的，并提供了调试出错的推理路径的机会。最后，研究人员可以在多种复杂任务中使用思维链推理，比如数学应用题、常识推理、符号运算等，总的来说，任何可以通过人类语言解决的任务都可以尝试使用思维链进行推理增强。

1.2.2 基于预训练的大语言模型推理能力增强

随着人工智能的发展，大型语言模型对序列信息建模的能力有目共睹，创建了像 ChatGPT、GPT-4 这样惊人的产品。代码生成能力作为衡量大语言模型推理能力的重要指标，也逐渐收到更多研究人员和研究机构的重视，通过提高大语言模型代码生成能力来增强其推理能力的方法也日益成为研究的主流。软件 3.0 的图景希望 AI 作为操作系统可以直接控制硬件，让程序员可以解放双手去编写驱动、操作系统和软件或是研发新的硬件。为了实现该愿景，越来越多

的针对代码生成进行增强预训练的大语言模型被开发出来。目前主流的代码生成模型主要是从两个方向进行训练的，即从基础模型进行代码预训练和仅针对代码进行预训练，下面本文将分别介绍这两种模式。

第一种预训练模式是使用基础大语言模型进行代码预训练。CodeX 是 Mark Chen 等人 [11] 在 GPT-3 的基础上使用代码数据进行了 Fine-Tuning 后获得的针对代码生成增强的大语言模型，CodeX 在 GPT-3 的基础上采用 175 步线性预热和余弦学习率衰减训练了总共 1000 亿个标记，微调后的 CodeX 在 HumanEval 数据集上取得了 28.8% 的通过率，对于同一个数据集，GPT-3 的通过率接近于 0。Code Llama 是 Baptiste Rozière 等人 [12] 从基于通用文本和代码数据进行预训练的基础模型 Llama 2 的基础上训练的模型。在初始设计阶段，研究者从 Llama 2 的 7B、13B、34B 和 70B 版本开始，在 500B 个 token 上训练 Code Llama 7B、13B 和 34B，在 1T 个 token 上训练 Code Llama 70B。为了应对代码生成问题中出现的长题干理解问题，研究者们在 Code Llama 中设计了一个专用的长上下文微调 LCFT（Long-Context Fine-Tuning）结构，在结构中模型使用了 16,384 个被标记的序列，高于 Llama 2 使用的 4,096 个序列。基于此 Code Llama 模型不仅在微调期间使用的序列长度内有效，而且进一步显示出外推能力并在多达 100,000 个标记的超长序列上表现出稳定的行为。在提高生成代码正确率方面，Code Llama 使用 AdamW 作为优化器，使用具有 1000 个预热步骤的余弦计划进行预训练，最终使模型在多个代码基准测试中达到了开放模型中最先进的性能，在 HumanEval 和 MBPP 上的得分分别高达 67% 和 65%。与 GPT-3、PaLM 和 CodeX 类似，Qinkai Zheng 等人 [13] 设计的 CodeGeeX 模型也遵循生成式预训练架构 Generative Pre-trained Transformer (GPT)，采用 Decoder-Only 风格进行自回归语言建模。这是一个具有 130 亿个参数的多语言代码生成模型，该模型可以迭代地将代码 token 作为输入，并在预测下一个 token 的同时将其与问题领域知识进行比较。截至 2022 年 6 月，CodeGeeX 已在 23 种编程语言的 8500 亿个 token 上进行了预训练，研究表明，CodeGeeX 可以帮助 83.4% 的用户提高编码效率。

第二种预训练模式是仅针对代码进行预训练。基础模型虽然已经展示了令人印象深刻的代码生成能力，能够完成简单的编程任务。然而，这些模型在对更复杂、描述更模糊的问题进行推理时仍然表现不佳，例如，基础模型在面对需要理解算法和复杂题干的竞赛级编程问题上普遍得分较低。这些问题要求模型具有理解推理问题的能力，而不仅仅是将指令转换成代码。为了弥补

这一差距，研究者们尝试了仅使用代码进行预训练的方法来生成新的大语言模型，从而获得具有更强代码生成能力和推理能力的新模型。AlphaCode 作为 Yujia[14] 等人设计的一个代码生成系统，其提出就是为了用来解决竞赛级的编程问题。研究者利用大型的 Transformer 语言模型生成代码，在选定的 GitHub 代码上对其进行预训练，经过过滤，最终的预训练数据集包含了总计 715.1GB 的代码。研究人员评价了 AlphaCode 在流行的 Codeforces 平台上举办的模拟编程竞赛中的代码推理生成效果，结果表面，在最近 10 场每场超过 5,000 名参与者的竞赛中，AlphaCode 的平均排名在前 54.3% 。基于这些结果，研究人员估计 AlphaCode 系统至少可以在过去 6 个月内参加过比赛的用户中排名前 28%，这是大语言模型首次在编程竞赛中获取如此高的成绩。通过研究人员对系统的进一步分析，证明了 AlphaCode 不是通过从训练数据集中复制代码段来解决问题，而是能够依赖自然语言问题描述来创建解决方案。Daniel Fried 等人 [15] 设计的 INCODER 模型使用了来自 GitHub 和 GitLab 上的具有非版权的和开源许可证的公共代码语料库和来自 StackOverflow 的问题、答案和评论构建的训练数据集。最终的预训练语料库共包含 159 GB 的代码，其中 52 GB 是 Python 代码，另外 57 GB 的内容来自 StackOverflow，这使得 INCODER 成为了第一个能够填充任意代码区域的大型代码生成模型。与 LLaMA 类似，Raymond Li 等人 [16] 设计的 StarCoder 是一个基于 1 万亿个词元训练的一个约 15B 参数的模型。此外，该模型还针对一个 35B 词元的 Python 数据集进行了微调，研究人员发现 StarCoderBase 在流行的编程基准测试中表现优于其他现有的开源的代码大语言模型，同时与 OpenAI 的 ‘code-cushman-001‘ 等闭源模型相比，其推理能力也相当甚至超过了这些闭源模型。凭借超过 8,000 个词元的上下文长度，StarCoder 模型可以处理比任何其他开源大语言模型更多的输入，这也让 StarCoder 可以应用于更多的更广泛的领域。例如，通过用多轮对话来提示 StarCoder 模型，软件开发人员可以让 StarCoder 充当技术助理。此外，该模型还可用于自动补全代码、根据指令修改代码以及用自然语言解释代码片段等任务。

然而，基于预训练的大语言模型推理能力增强依然面临几个挑战：首先是不论基于何种模型进行预训练，都需要投入巨大的时间成本和人力成本。此外，随着问题需求的逐渐多样化，基于预训练的大模型无法根据多变的需求灵活的进行问题处理。其次，仅依靠训练后模型的自身能力和参数规模进行推理，难以确定推理流程，也无针对推理过程中的某些节点进行单独修改 [17]。

为了促进解决上述挑战并充分利用思维链和大语言模型预训练系统的优

势，本研究提出了一个基于算法模板的大语言模型代码生成方法。该方法旨在结合思维链技术的高效性和针对性预训练模型在代码生成上的强大能力，综合提高模型的代码生成效果。在代码生成时，该方法首先利用问题解析器提取问题的关键信息，然后在此基础上对于构建算法模版进行向量匹配，以得到更加精细化的推理过程。通过问题解析进行问题信息提取，能最大程度的模拟人脑的思维过程，并充分利用推理过程中的历史信息；同时引入算法模板，增强对大语言模型推理的准确率。

1.3 本文贡献

为了验证本研究提出的算法模板在大语言模型代码生成方面的效果，本文在两套不同的数据集上进行代码生成和验证实验，包括 HumanEval-X[13]，一套由清华大学构建的一个测试大语言模型代码生成能力的数据集，HumanEval-X 包含 820 个高质量手写样本，覆盖多种语言和多种任务类型。以及笔者自行整理的来自 LeetCode 上不同类型的数百道算法题，这些题都具有极强的代表性，可以包含常用的所有算法类型。本文采用 CodeX 所使用的无偏 pass@ k[18] 指标，判断生成代码的功能正确性。实验的设计旨在评估 ATI 方法在大语言模型代码生成上产生的性能差异，验证其有效性。本研究的主要贡献概括如下：首先，本文结合已有文献和调研，深入分析了在真实场景中使用现有的大语言模型推理能力的局限性，特别是在生成真实项目的复杂代码时呈现出的推理能力不足的问题。其次，本文提出了 ATI 方法，这是一种结合问题解析器和算法模板库的新型的代码生成增强方法。通过引入多阶段的推理和检索流程，该方法不仅模拟了真实的人脑思维模式，还显著提升了推理效率，得到优于原始模型的实验效果。最后，我们还探讨了问题解析器和算法模板库在大语言模型代码生成任务中的有效融合策略，为后续相关研究提供了新的视角和启示。实验结果对比分析表明，这一结果也充分显示 ATI 方法在大语言模型代码生成中的优势，同时也展示了将问题解析器和算法模板库用于大语言模型代码生成任务上的巨大潜力。

1.4 本文组织结构

本文的结构划分如下：

第一章，引言。本章节首先阐述了增强大语言模型代码生成能力的意义和背景。近年来，自动代码生成技术在软件开发中的应用越来越广泛，但现有的大语言模型在处理复杂编程任务时仍存在诸多挑战。接着，在研究现状部分介绍了现有的增强大语言模型的代码生成能力的方法，以及相关研究的演变趋势。最后，本章总结了本文工作的主要内容和贡献。

第二章，相关技术。该章节介绍了增强大语言模型代码生成能力的相关技术，包括检索增强生成技术、文本嵌入模型以及基于注意力机制的 Transformer 模型和它们在大语言模型代码生成中的应用。

第三章，基于算法模板的大语言模型代码生成方法。本章节首先介绍了本文的研究内容，并对相关技术面临的问题和国内外对于该问题的研究现状进行了分析和总结。接着，本章节阐述了我们设计的算法模板库方法的概念和应用步骤，详细介绍了 ATI 方法核心的问题解析阶段和算法模板匹配生成阶段。

第四章，实验设计与结果分析。本章节首先介绍了我们的实验设计，阐述了我们选择基准模型和基准方法的理由。然后，我们介绍本文实验使用的实验数据以及对实验结果采用的评估指标。最后，本章对于实验结果进行了详细的分析，并设计了消融实验，进一步验证我们实验的有效性。

第五章，总结与展望。本章节总结了本文的主要研究工作，并结合文献与近期热门研究趋势对未来可能的研究方向进行展望。总结部分重申了基于算法模板来增强大语言模型代码生成能力的优势，并指出当前研究的不足和未来的改进方向。展望部分提出了几种可能的研究方向，如进一步优化算法模板的设计、更高效的参数调优方法，以及在更多编程语言和应用场景中的推广和应用。

第二章　相关技术

本章主要介绍检索增强生成 RAG 与 Transformer 领域的技术应用和发展。检索增强生成主要是用来帮助大模型提高其面对复杂问题的解决能力。Transformer 模型的发展进一步规范了大语言模型的发展方向。本章首先回顾了检索增强生成的相关技术，然后阐述了 Transformer 模型的相关基础理论知识。最后从不同角度介绍了与两者相关的技术和知识背景。

2.1　检索增强生成技术

在大语言模型的应用中，研究人员常面临众多挑战，包括领域知识的缺乏，因为模型自身的知识完全源于它的训练数据，而现有的主流大模型的训练集基本都是构建于网络公开的数据，对于一些实时性的、非公开的或离线的数据是无法获取到的，这部分知识也就无从具备；幻觉问题，所有的 AI 模型的底层原理都是基于概率统计的，其模型输出实质上是一系列数值运算，大模型也不例外，这就导致模型可能会输出一些严格符合输出格式但是混乱的输出内容，尤其是在大模型自身不具备某一方面的知识或不擅长的场景。而这种幻觉问题的区分是比较困难的，因为它要求使用者自身具备相应领域的知识；最后是数据安全问题，对于企业来说，数据安全至关重要，没有企业愿意承担数据泄露的风险来将自身的私域数据上传第三方平台进行训练。这也导致在使用大语言模型生成解决方案时不得不在数据安全和效果方面进行取舍。检索增强生成 RAG（Retrieval-Augmented Generation）[19] 通过引入外部知识库等额外信息源，为这些问题提供了有效的解决策略。RAG 在那些需要不断更新知识的知识密集型场景或特定领域应用中尤为有效。与其他方法相比，RAG 的一大优势是无需针对特定任务重新训练大语言模型。RAG 是 2023 年基于大语言模型的系统中最受欢迎的架构。许多产品基于 RAG 构建，从基于 WEB 搜索引擎和大语言模型的问答服务到使用私有数据的 CHAT 应用程序。RAG 的研究不仅优化了预训练方法，还逐渐融合了 ChatGPT 和 Mixtral 这样的高性能微调模型的优势。下面是 RAG 应用的典型工作流程：

- 输入：即大语言模型系统需要回答的问题。如果不使用 RAG，问题直接由大语言模型回答。
- 索引：使用 RAG 时，会先将相关文档分块，为这些块生成嵌入向量，并将它们索引到向量库中。在进行查询时，查询内容也会以相似的方式进行嵌入。
- 检索：通过比较查询内容与索引向量，找到相关的文档。
- 生成：将找到的相关文档与原始提示结合作为额外上下文，然后传递给模型进行回应生成，最终形成系统对用户的回答。

RAG 的架构如图 2-1，简单来讲，RAG 就是通过检索获取相关的知识并将其融入 Prompt，让大模型能够参考相应的知识从而给出合理回答。因此，可以将 RAG 的核心理解为“检索 + 生成”[19]，前者主要是利用向量数据库的高效存储和检索能力，召回目标知识；后者则是利用大模型和 Prompt 工程，将召回的知识合理利用，生成目标答案。近几年来，RAG 系统经历了从初级阶段到高级阶段，再到模块化阶段的演变。这一进化过程旨在帮助大语言模型克服性能、成本和效率等方面的挑战。

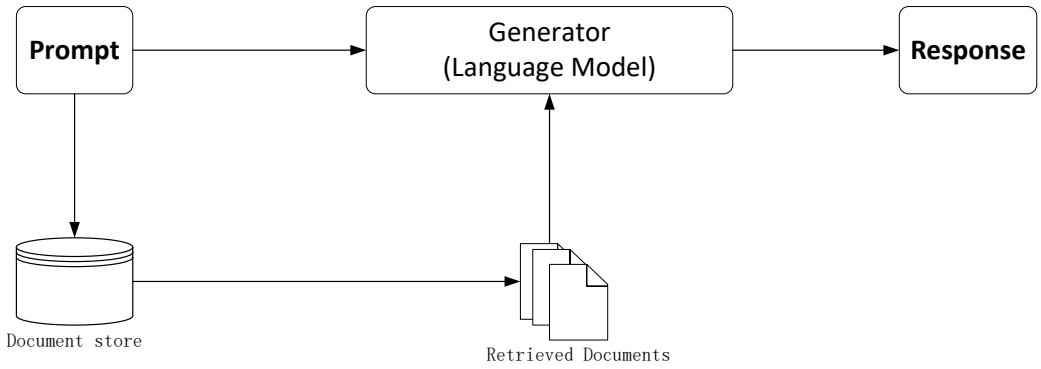

图 2-1: RAG 架构

2.1.1 初级 RAG

初级 RAG 基于一套简单的流程，包括索引建立、文档检索和内容生成。总得来说，系统根据用户的输入查询相关文档，然后将这些文档和一个提示语结合起来，交给模型生成最终的回答。如果涉及到多轮对话，还可以将对话历史整合到提示语中。基于此，一个完整的 RAG 应用流程可以被概括为两个阶段：

- 数据准备阶段：包括数据提取、文本分割、向量化（Embedding）以及数据入库。

- 应用阶段：包括用户提问、数据检索、注入 Prompt 和最后的大语言模型生成答案。

数据准备阶段是初级 RAG 的重点。数据准备一般是一个离线的过程，其任务是将私域数据向量化后构建索引并存入数据库。主要包括：数据提取、文本分割、向量化、数据入库等环节。数据提取包括数据加载，使用多格式数据加载和不同数据源获取等，根据数据自身情况，将数据处理为同一个范式；数据处理，对数据进行过滤、压缩、格式化等；元数据获取，提取数据中关键信息，例如文件名、标题、时间等信息。文本分割主要考虑两个关键因素，即 Embedding 模型的 Tokens 限制情况和语义完整性对整体的检索效果的影响。主流的文本分割方法包括句分割，将文本以”句”的粒度进行切分，保留一个句子的完整语义，常见切分符包括：句号、感叹号、问号、换行符等；固定长度分割，根据 Embedding 模型的 token 长度限制，将文本分割为固定长度，这种切分方式会损失很多语义信息，需要在头尾增加一定冗余量来缓解。向量化是一个将文本数据转化为向量矩阵的过程，该过程会直接影响到后续检索的效果。目前常见的 Embedding 模型能满足大部分需求，但对于特殊场景（例如涉及一些罕见专有词或字等）或者想进一步优化效果，则需要选择闭源的 Embedding 模型微调或直接训练适合自己场景的 Embedding 模型。数据入库是对向量化后的数据构建索引，并写入数据库的过程，适用于 RAG 场景的数据库包括：FAISS、Chromadb、ES、milvus 等。

初级 RAG 的局限性包括低精确度（检索到的信息不够准确）和低召回率（有时候无法检索到所有相关的信息）。此外，有时候模型可能会接收到过时的信息，这正是 RAG 系统希望首先解决的问题之一。这可能会导致模型产生不基于事实的幻想性回答，从而影响回答的准确性和可靠性。当引入额外信息以增强回答时，还可能出现信息重复或冗余的问题。处理多个检索到的文档时，如何排列它们的优先级以及如何使生成的内容风格和语调一致也是需要考虑的挑战。我们还需要确保生成的任务不会过分依赖于这些额外信息，避免模型仅仅重复这些信息而缺乏创新。

2.1.2 高级 RAG

高级 RAG [20] 解决了初级 RAG 面临的问题，尤其是在提高检索质量方面，包括优化检索前、检索时和检索后的各个过程。在检索前的准备阶段，研究人员们通过优化数据的索引建立来提高数据质量，包括改善数据的细节度、优化索引结构、添加元数据、改进对齐方式以及混合检索方法。在检索阶段，研究人员们通过改进嵌入模型来提高上下文片段的质量。例如，通过对嵌入模型进行微调，以提高检索的相关性，或者使用能够更好理解上下文的动态嵌入模型。在检索后的优化过程中，研究人员们专注于解决上下文窗口限制和减少噪音或分散注意力的信息。常用的方法包括重新排列文档，以将更相关的内容放在提示的前后，或者重新计算查询与文档片段之间的语义相似度。此外，通过压缩提示信息也有助于解决这些问题。

2.1.3 模块化 RAG

模块化 RAG[21]，顾名思义，通过增强其功能模块来提升性能，例如加入相似性检索的搜索模块，以及在检索工具上进行精细调整。模块化 RAG 能够根据具体的任务需求，添加、替换或调整模块之间的工作流程，从而实现更高的多样性和灵活性。这种设计让模块化 RAG 不仅包括了初级 RAG 和高级 RAG 这两种固定模式，还扩展了包括搜索、记忆、融合、路由、预测和任务适配等多种模块，以解决各种问题。随着模块化 RAG 系统构建变得更加灵活，一系列优化技术相继被提出，用于进一步优化 RAG 流程，包括：

- 混合式搜索探索：结合了关键词搜索与语义搜索等多种搜索技术，以便检索到既相关又富含上下文的信息，特别适用于处理多样化的查询类型和信息需求。
- 递归式检索与查询引擎：通过从小的语义片段开始，逐步检索更大的内容块以丰富上下文的递归过程，有效平衡了检索效率与信息的丰富度。
- StepBack-prompt 提示技术：一种特殊的提示方法，能让大语言模型进行概念和原则的抽象化处理，从而引导更加深入的推理过程。当应用于 RAG 框架时，能够帮助模型超越具体事例，进行更广泛的推理。
- 子查询策略：采用树状查询或按序查询小块信息的策略，适用于多种场景。LlamaIndex 提供的子问题查询引擎允许将大的查询任务拆分成多个小问题，分别利用不同的数据源进行解答。

- 假设性文档嵌入技术（HyDE）[22]：通过生成查询的假设性回答并嵌入，来检索与这个假设回答相似的文档，而不是直接使用查询本身，以此来优化检索效果。

综上研究，我们看到了 RAG 系统的快速进步，这包括了能够定制并推动 RAG 在多个领域性能和实用性进一步提高的先进范式的开发以及用于评估 RAG 性能的更优秀的评估工具和衡量指标的研究。同时，伴随着大语言模型应用的逐步广泛，基于 RAG 思想的文本嵌入模型也迎来了蓬勃发展，

2.2 文本嵌入模型技术

检索增强生成技术在大语言模型中的主要应用形式就是文本嵌入模型，它将文本数据编码到潜在空间中，其中数据的底层语义可以通过输出嵌入来表达。随着预训练语言模型的出现，文本嵌入的质量得到了显着提高，使其成为信息检索 IR（Information Retrieval）的必不可少的组件。文本嵌入模型能够将非结构化的信息转化为结构化的信息，这样就可以把人类能够理解的文字转变为计算机可以分析的数据，基于这些大语言模型就可以完成文字分类、情绪判断等我们日常工作中常做的任务。这也表明了文本嵌入模型并不是简单的文本匹配，而是通过向量分析从语义层面来考察文本的相似性。文本嵌入模型强大的匹配让它可以给多种领域提供支持，从聚类和主题表示到搜索系统和文本挖掘再到下游模型的特征表示。通常在这些应用中使用文本嵌入模型会导致计算量成倍的增加，但具体带来了多少效果增强确实难以量化的，这导致一些模型被“盲目”应用于行业中的新用例。为了评估这种相似性，Niklas Muennighoff 等人 [23] 提出了海量文本语义向量基准测试 MTEB（Massive Text Embedding Benchmark）。MTEB 旨在阐明模型在各种嵌入任务中的表现，从而构建一个评价文本嵌入模型性能的基准。MTEB 由 58 个数据集组成，涵盖 8 个嵌入任务中的 112 种语言：双语挖掘、分类、聚类、配对分类、重新排序、检索、STS 和摘要。MTEB 软件是开源的，只需添加不到 10 行代码即可评估任何嵌入模型，MTEB 的出现使文本嵌入模型的结果变为可控的和可测量的。随着深度学习技术的发展，文本嵌入模型被广泛应用在信息检索领域。基于文本嵌入的检索方法通过将文本数据转换为高维向量，以捕捉文本的语义信息，从而实现高效的信息检索。研究者们通常利用密集嵌入、稀疏嵌入和多向量方法来增强信息检索效果，本文将从这三个角度介绍当前的研究现状。

2.2.1 稀疏嵌入

稀疏嵌入（Sparse Embeddings）[24] 方法通过生成稀疏向量来表示文本，通常包含大量的零值，它们通过统计词频和逆文档频率来计算词汇的重要性，从而生成稀疏的文本表示。Robertson[25] 等人提出的 BM25 模型，BM25 是一种用于信息检索和文本挖掘的算法，它被广泛应用于搜索引擎和相关领域。BM25 基于 TF-IDF 的思想，TF-IDF 的核心思想是：如果一个词语在某一文档中频繁出现，但在其他文档中较少出现，则认为该词语对于该文档具有较高的重要性。TF-IDF 由两部分组成：一是词频 TF（TermFrequency），二是逆文档频率 IDF（Inverse Document Frequency）。其中词频 TF 是指给定词语在文档中出现的频率。由于长文档中同一词语的出现次数通常会多于短文档，因此在计算词频时会进行归一化处理，以消除文档长度的影响。词频的计算方法如公式（2–1）所示。

$$\mathrm{TF}_{i,j} = \frac{n_{i,j}}{\sum n_{k,j}} \tag{2–1}$$

其中，n_{ij} 是词语 i 在文档 j 中的出现次数，分母是文档 j 中所有词语出现次数的总和。逆文档频率 IDF 衡量的是一个词语在所有文档中的普遍重要性。包含给定词语的文档数越多，逆文档频率值越小。逆文档频率的计算如公式（2–2）所示。

$$\mathrm{IDF}_i = \log\left(\frac{|D|}{1 + |d_i|}\right) \tag{2–2}$$

其中，$|D|$ 是文档集合中文档的总数，$|d_i|$ 是包含词语 i 的文档数目，分母加 1 是为了避免除数为零。

综合词频和逆文档频率，我们可以按照公式（2–3）计算出 TF-IDF 的值，用于评估词语在文档中的重要性。

$$\text{TF-IDF}_{i,j} = \mathrm{TF}_{i,j} \times \mathrm{IDF}_i \tag{2–3}$$

BM25 算法通过对文档中的每个词项引入饱和函数（saturation function）和文档长度因子，改进了 TF-IDF 的计算。其计算方法如公式（2–4）所示

$$\mathrm{BM25}(D, Q) = \sum_{i=1}^{n} \mathrm{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{\mathrm{len}(D)}{\mathrm{avg_len}}\right)} \tag{2–4}$$

其中，n 是查询中的词项数，$|q_i|$ 是查询中的第 i 个词项，$\mathrm{IDF}(q_i)$ 是逆文档

频率，计算方式如公式（2-3），$f(q_i, D)$ 是词项 $|q_i|$ 在文档 D 中的出现次数（TF），len(D) 是文档 D 的长度，avg_len 是所有文档的平均长度，k_1 和 b 是调整参数。BM25 通过对词频和文档长度进行归一化处理，提高了检索的准确性。在现代信息检索系统中，BM25 仍然是一个重要的基线模型，被广泛应用于各种信息检索任务中。这些方法虽然简单，但在处理大规模文档集时效率较高，且在一些任务中表现仍然优越。

2.2.2 密集嵌入

密集嵌入（Dense Embeddings）[26, 27] 方法通过将文本表示为固定长度的高维向量来捕捉其语义信息。这类方法通常使用深度神经网络，如 BERT、GPT 等，通过大规模语料库进行预训练，以生成具有强大语义理解能力的嵌入表示。在实际应用中，密集嵌入可以有效地捕捉文本的语义相似度，Devlin[28] 等人提出的 BERT（Bidirectional Encoder Representations from Transformers）模型，BERT 是一种自然语言处理领域中的文本表征模型，在 Transformer 架构的基础上，经过大量的预训练与微调，并结合掩蔽语言模型（Masked Language Model, MLM）和下一句预测（Next Sentence Prediction, NSP），让其能够捕捉上下文中的双向信息，使得生成的文本嵌入更具有丰富的语义信息。BERT 模型的构建分为两个步骤：

- 预训练：在预训练期间，模型在不同的预训练任务上对未标记数据进行训练。BERT 并不是使用传统的从左到右或从右到左的语言模型来预训练 BERT。相反，而是使用无监督任务进行预训练。在训练过程中，Devlin 等人会随机屏蔽一定比例的输入标记，然后让模型去预测这些被屏蔽的标记，这种被称为 MLM 的过程赋予了 BERT 深度双向表示的能力
- 微调：对于微调，首先使用预训练的参数初始化 BERT 模型，然后使用来自下游任务的标记数据对所有参数进行微调。即使模型是用相同的预训练参数初始化的，但具体到下游任务时，每个下游任务都会有自己微调模型。但是比起预训练来说，微调还是要更加简单，因为 Transformer 中的自注意力机制允许 BERT 通过交换适当的输入和输出来建模许多下游任务。对于涉及文本对的应用程序，一种常见的模式是在应用双向交叉注意力之前独立编码文本对。但 BERT 开创性的使用自注意力机制将这两个阶段统一起来，因为通过自注意力对连接的文本对进行编码的过程，实际上考察的是两个句子之间的双向交叉注意力。对于每个任务，我们只需将特定于任务的输

入和输出插入 BERT ，然后端到端的微调所有参数即可。

在信息检索任务中，BERT 的应用极大地提升了检索效果，尤其是在需要理解复杂语义的场景下，这使得大语言模型在问答系统、文本分类和相似度检索等任务中表现出色。

2.2.3 多向量

多向量（Multi-vector）[29] 方法则通过将文本表示为多个向量，以更好地捕捉文本中的多种语义信息。这类方法能够在更细粒度的层面上进行文本表示，使得信息检索系统能够更准确地匹配用户查询和文档内容。Khattab 和 Zaharia 等人 [24] 提出的 ColBERT 模型，ColBERT 全称为（Contextualized Late Interaction over BERT），基于传统的 BERT 模型进行了深度创新。BERT 将 token 向量合并为单一表示，而 ColBERT 保留了每个 token 的表示，提供了更细粒度的相似性计算。ColBERT 的独特之处在于引入了一种新颖的后期交互机制，可以通过在检索过程的最终阶段之前分别处理查询和文档，实现高效和精确的排名和检索。本质上，虽然 BERT 或其他传统的 Embedding 模型都可以为每个文档生成一个单一向量，并通过产生一个单一的数值分数来反映其与查询句的相关性。但 ColBERT 提供了一个向量列表，通过在检索阶段对每个词生成一个向量，使得检索系统能够在查询和文档之间进行精细的词级别匹配，从而提升检索效果。具体实现为：对于每个查询嵌入任务，ColBERT 都通过 MaxSim 运算符与所有文档嵌入进行交互，该运算符计算最大相似度（例如余弦相似度），并且这些运算符的标量输出都是在查询词之间求和获得的。此外，它还使得 ColBERT 能够利用向量相似性搜索索引 [30, 31] 直接从大型文档集合中检索前 k 个结果，与仅对基于术语的检索输出进行重新排名的模型相比，ColBERT 的召回率大幅提高。这种方法也可以帮助我们更详细和更细致的理解查询和文档之间的语义关系。

如图2–2所示，ColBERT 模型结构包括查询编码器 f_Q、文档编码器 f_D 以及后期交互机制。给定查询 q 和文档 d，f_Q 将 q 编码为一个固定大小的嵌入包 E_q，而 f_Q 将 d 编码为另一个包 E_d。重要的是，E_q 和 E_d 中的每个嵌入分别基于 q 或 d 中的术语进行了语境化。使用 E_q 和 E_d，ColBERT 通过后期交互计算 q 和 d 之间的相关性得分，这就是 MaxSim 运算后的总和。具体来说，我们找到每个 $v \in E_q$ 与 E_d 中的向量的最大余弦相似度，并通过求和合并输出。除了余弦之外，我们还评估了 L2 的平方距离作为向量相似度的度量。虽然使用其

他选择（例如深度卷积和注意力层）可以实现更复杂的匹配，但最大相似度计算的总和具有两个鲜明的特征。首先，它是一种特别便宜的交互机制。其次，更重要的是，它可以实现高效的 Top-k 检索剪枝，它允许使用向量相似度算法跳过文档，而无需实现完整的交互矩阵，甚至无需单独考虑每个文档。其他与 ColBERT 同样成本的匹配方法都难以实现高效的剪枝。

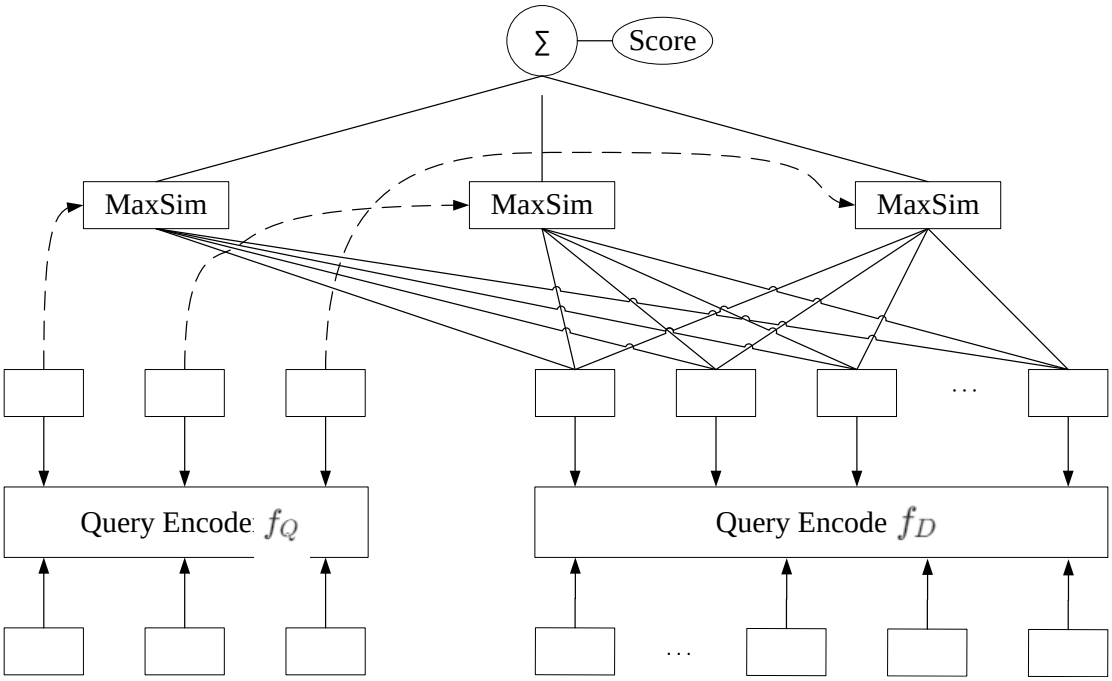

图 2–2: colbert 结构模型

ColBERT 可以利用深度语言模型的表现力，同时获得预先计算离线文档表示的能力，从而大大加快查询处理速度。除了降低重新排序传统模型检索到的文档的成本之外，ColBERT 的修剪友好型交互机制还能够利用向量相似性索引直接从大型文档集合中进行端到端检索。同时，ColBERT 模型在大规模文本检索任务中也表现出色，尤其是在需要高精度检索的应用场景下多向量方法通常结合了密集和稀疏嵌入的优点，进一步提高检索性能。

综上所述，基于文本嵌入模型的信息检索方法在密集嵌入、稀疏嵌入和多向量三个方面都有了显著的发展。密集嵌入通过深度神经网络捕捉文本语义，稀疏嵌入通过统计方法提高检索效率，而多向量方法则结合两者优势，提升了检索的精细度和准确性。这些方法的不断进步，推动了信息检索技术的发展，为各类应用提供了有力的技术支持。

2.3 思维增强方法

大多数复杂任务都包含隐式约束、复杂对象关系以及上下文中错综复杂的变量和参数。因此，在推理阶段，大语言模型需要克服三个主要挑战：提取重要信息、识别潜在约束和执行准确推理 [32]。这些挑战将给单个大语言模型带来沉重的负担。为了解决这些挑战，研究人员们设计出了辅助大语言模型推理的思维增强方法。最开始，研究人员们使用经典的 CoT 来解决涉及数学、常识或符号推理的广泛任务。CoT 可以显著提高大语言模型解决问题的能力，而无需诉诸任何模型更新，CoT 的模型结构如图2–3所示。CoT 的一项重大改进是具有自洽性的 CoT （CoT-SC）[33]，该方法会生成多个 CoT 方案，然后选择最佳 CoT 作为结果。

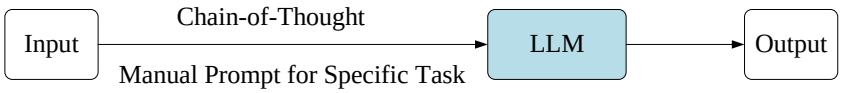

图 2–3: CoT 结构模型

尽管大语言模型表现出来很擅长这种逐步分解，但即使问题分解正确，大语言模型也经常在生成解决方案部分犯逻辑和算术错误。Luyu Gao[34] 等人在 CoT 模型的基础上进一步设计了程序辅助语言模型 PAL。它使用大语言模型读取自然语言问题并生成 Python 程序作为中间推理步骤，最后将解决方案装载到 Python 解释器上运行。在该系统中，大语言模型的唯一任务就是使用 PAL 将自然语言问题分解为可运行的步骤，具体的问题解决则交由程序解释器运行。该模型的提出得益于大语言模型能力的发展，目前这些排名前列的大语言模型都已经在自然语言和编程语言上进行了预训练 [35–37]。PAL 方法创新性的将代码作为“小样本提示”加入到提示链中，并且限制了大语言模型的工作，让其能专注于推理过程而不是运行过程。基于这种创新，PAL 很好的解决了 CoT 推理中常出现的推理链可能是正确的，但会产生错误答案的问题，一个具体的 PAL 提示案例如图2–4所示，其中思维链推理以蓝色突出显示，PAL 步骤以红色突出显示，Python 解释器运行以绿色突出显示。

CoT 等单链推理模型专注的是对问题的逐步分解，这种从左到右的推理过程虽然简单高效，但是也常常局限于模型的 token 量级，并且在需要探索、战略前瞻或初始决策起关键性作用的任务上表现不佳 [38]。为了解决这些问题，Shunyu Yao[9] 等人设计了一种基于树形的大语言模型推理框架 ToT，ToT 允许

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 tennis balls.
tennis_balls = 5
2 cans of 3 tennis balls each is
bought_balls = 2 * 3
tennis balls. The answer is
answer = tennis_balls + bought_balls

Q: The bakers at the Beverly Hills Bakery baked 200 loaves of bread on Monday morning. They sold 93 loaves in the morning and 39 loaves in the afternoon. A grocery store returned 6 unsold loaves. How many loaves of bread did they have left?

Model Output

A: The bakers started with 200 loaves
loaves_baked = 200
They sold 93 in the morning and 39 in the afternoon
loaves_sold_morning = 93
loaves_sold_afternoon = 39
The grocery store returned 6 loaves.
loaves_returned = 6
The answer is
answer = loaves_baked - loaves_sold_morning - loaves_sold_afternoon + loaves_returned

```
>>> print(answer)
74
```

图 2-4: PAL 模型示例

大语言模型通过考虑多种不同的推理路径并对这些路径进行自我评估来决定下一步行动，以及在必要时向前看或回溯以做出全局选择，进而作出深思熟虑的决策。这种多查询推理的方法若让大语言模型可以在返回的多条推理方法中进行评估和选择，这也让大语言模型的推理过程更接近人类思维。ToT 模型和 CoT 模型的对比如图 2-5所示

Maciej Besta 等人 [10] 认为，通过将严格的推理结构强加于思维过程反而会从根本上限制了提示中的推理能力。因此他们将大语言模型思维设计成任意图形结构 GoT，从根本上实现更强大的提示。这也更加符合人类的思维过程，人类的任务解决通常是非线性的，它涉及到将中间的解决方案组合成最终的解

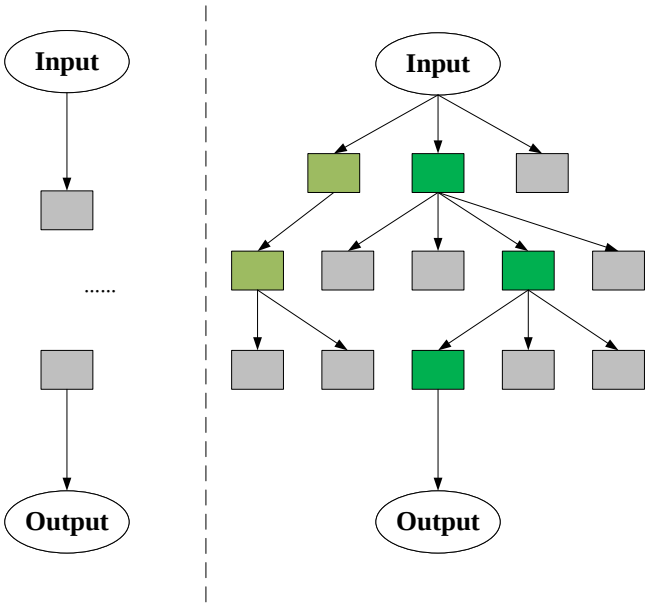

图 2-5: ToT 和 CoT 的对比，每个矩形框代表一个推理想法

决方案，或者在发现新的见解时改变推理的流程。得益于其全面的推理流程，GoT 模型及其变种相关被广泛的用于增强大模型推理能力方面。同时使用 GoT 设计思维链可以获得更大的思想体积，并让构建快速工程的工作变得更加便利，GoT 模型结构如图 2-6所示。

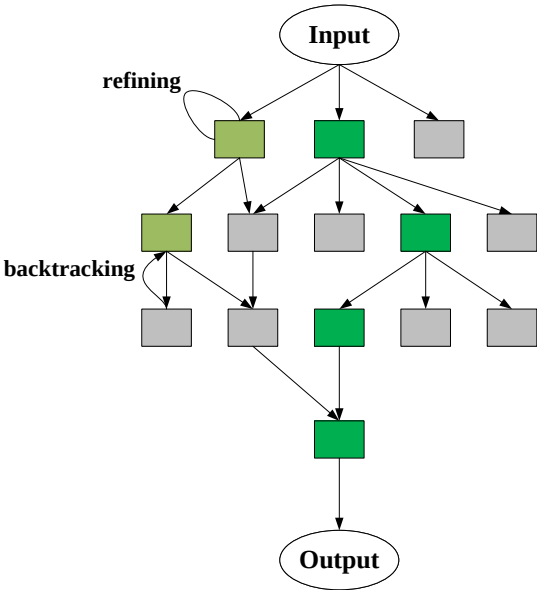

图 2-6: GoT 模型结构

2.4 Transformer 模型

在自然语言处理领域，Akram Elghouat 等人 [39] 设计的 Transformer 模型已经成为主流的深度学习架构。Transformer 是一种基于自注意力机制（Self-Attention Mechanism）和前馈神经网络（Feed-Forward Network, FFN）的深度学习模型，其大量被应用于自然语言处理领域，并且在机器翻译问题上表现出了很高的性能可靠性。Transformer 在设计上并不是依赖传统的卷积神经网络（CNN）结构和循环神经网络（RNN）结构，而是加入了自注意力机制和前馈神经网络机制，使得 Transformer 模型拥有了并行处理序列元素的能力，这极大的提高了大语言模型的训练效率。Transformer 模型由两个部分组成：编码器（Encoder）和解码器（Decoder）。

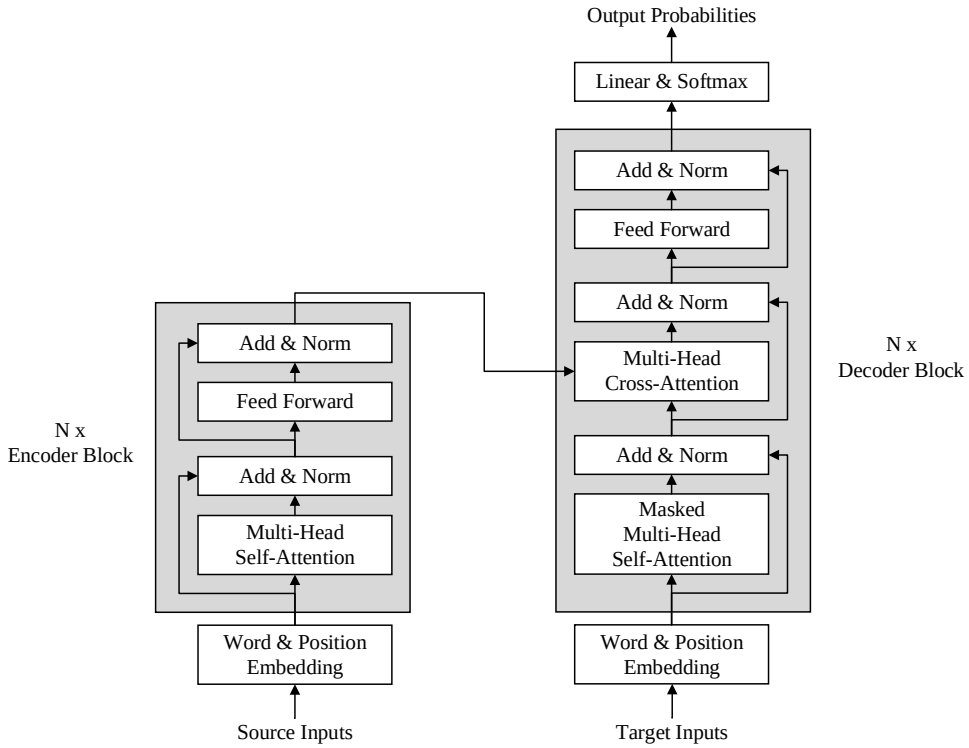

图 2-7: Transformers 架构示意图

2.4.1 编码器（Encoder）

编码器是 Transformer 模型重要组成部分，它的主要任务是捕捉输入序列的语义信息。在编码器中，每个输入词汇都会通过一个嵌入层（Embedding Layer）转换成固定维度的向量表示。Transformer 模型会利用多个自注意力层（Self-Attention Layer）和前馈神经网络（Feed-Foward Neural Network Layer）对这些向量进行处理，以捕捉词汇间的依赖关系和语义信息。Ashish Vaswani[39] 等人设计的编码器具有很多显著的优势，编码器可以通过自注意力机制来捕捉输入序列中的词汇间的长程依赖关系，这有助于理解句子的整体语义；编码器采用自注意力机制进行计算，相对于传统的循环神经网络 RNN（Recurrent Neural Network），这种计算方式更加高效，能够在处理长序列时避免梯度消失或梯度爆炸的问题。但同时，其依然具有一些不足之处，编码器无法直接处理变长的输入序列，这使得在处理不同长度的文本时需要进行截断或填充，可能会引入一些额外的误差。并且，编码器对于高频词汇的捕捉能力相对较弱，这可能导致模型对于常见词语的语义理解不够准确。

2.4.2 解码器（Decoder）

解码器是 Transformer 模型的核心部分，其主要任务是根据已处理好的输入数列生成新的输出数列。解码器接受编码器的输出序列，然后进行多轮预测，通过自专注层和前馈神经网络层产生新的输出序列。每一步预测都依赖于所有以前的预测结果，这使得解码器能够捕捉到更复杂的语言现象。与编码器一样，在使用过程中，Transformer 模型的解码器也展现出了自身的巨大优势和一些不足之处。由于解码器每一步的预测都依赖于所有以前的预测结果，因此它可以生成连贯的输出序列，这在许多 NLP 任务中是非常重要的。解码器可以通过自注意力机制捕捉到输入序列中每个词汇对当前输出的影响，从而更好地理解上下文信息。但同时，解码器的计算复杂度相对较高，因为它需要进行多轮预测，每轮预测都需要计算自注意力权重和前馈神经网络的输出。并且，由于解码器的训练是基于历史数据进行的，因此它可能无法很好地泛化到新的语言现象。这或许会造成某些特定场景下的表现乏善可陈。

在 Transformer 模型的编码器和解码器的设计过程中，上文提到的自注意力机制（Self-Attention Mechanism）和前馈神经网络（Feed-Forward Network, FFN）起到了至关重要的作用。

2.4.3 自注意力机制（Self-Attention Mechanism）

在传统的 RNN 循环神经网络和 LSTM（Long Short-Term Memory）长记忆网络中，输入序列都是有顺序的。这就意味着要处理好当前时间的步骤输入，往往需要依赖以往的计算结果，这种依赖可能会产生梯度消失或梯度爆发等问题。为了解决这些问题，Transformer 模型引入了自注意力机制（Self-Attention Mechanism），自注意力机制是通过对输入序列中各个位置的表示向量进行计算，然后根据这些表示向量计算出一个权重分布，最后在输入序列中的各个位置上应用这个权重分布，得到加权后的表示向量。在此基础上，Transformer 进一步提出了多头注意力机制，允许模型同时关注输入序列中的不同子空间，从而进一步提高了模型的表达能力。该机制允许模型直接处理当前输入的序列中的任意位置，而不需要依赖之前的计算结果。同时，自注意力机制可以让 Transformer 实现并行计算，使训练速度大大提高。

2.4.4 前馈神经网络（Feed-Forward Network）

在人工智能领域，通过模拟人的大脑神经元网络结构，研究人员们设计出了神经网络模型，它通过大量的神经元相互联系，可以对复杂的模型进行学习和识别。在 Transformer 模型中，前馈神经网络也扮演了重要角色。前馈神经网络是一种常见的神经网络结构。内含多个神经元层，各神经元从前一层接收输入讯号，向下一层传送输出讯号。在前馈神经网络中信息是从输入层层叠叠递送的，直至传到输出层得出最终结果为止。在 Transformer 中，由前馈神经网络先对输入序列进行线性变换，进而得到了一组新的表示向量，这些表示向量随后被用于计算注意力权重。这样，前馈神经网络就可以捕捉到输入序列中的语义信息。前馈神经网络给 Transformer 模型带来了极大的性能增强。得益于前馈神经网络简单的结构，其可以对输入序列进行快速响应，这使得 Transformer 在处理长序列时具有高效性，能够在短时间内得到结果。前馈神经网络可以在输入序列中捕捉到长程依赖，这使得 Transformer 在理解语义信息时更加准确，能够更好地捕捉到输入序列中的关键信息。前馈神经网络的训练也比较简单，这使得 Transformer 在训练过程中能够更快地收敛，并得到更好的结果。

经典的 Transformer 实现流程如图 2–7所示，对于一组输入序列 $(x_1, x_2, \ldots, x_n)$，其中 x_i 表示一个词。如公式 2–5所示，通对词嵌入向量进行位置编码来引入序

列中的位置信息，pos 是位置，i 是嵌入维度的索引，d 是词嵌入的维度。

$$\begin{aligned} PositionalEncoding(pos, 2i) &= \sin(\frac{pos}{10000^{2i/d}}) \\ PositionalEncoding(pos, 2i+1) &= \cos(\frac{pos}{10000^{2i/d}}) \end{aligned} \tag{2-5}$$

Transformer 使用多个自注意力层组成编码器结构，自注意力层使用的多头注意力机制如公式2-6所示，其中 d_k 是注意力头的维度，W_i^Q, W_i^K, W_i^V 是学习得到的权重矩阵，$Attention$ 是自注意力函数，$head_i$ 是每个注意力头的计算。

$$\begin{aligned} Attention(Q, K, V) &= softmax(\frac{QK^T}{\sqrt{d_k}})V \\ head_i &= Attention(Q \cdot W_i^Q, K \cdot W_i^K, V \cdot W_i^V) \\ MultiHead(Q, K, V) &= Concat(head_1, \ldots, head_h)W^O \end{aligned} \tag{2-6}$$

如公式2-7所示，Transformer 在计算完自注意力后会利用前馈神经网络对输入进行非线性变换。

$$PositionwiseFeedforward(x) = ReLU(xW_1 + b_1)W_2 + b_2 \tag{2-7}$$

同时，Transformer 会在每个自注意力层后进行归一化和残差连接。层归一化要求 Transformer 在每个子层之后使用层归一化，该方法有助于提高模型的训练稳定。残差连接要求 Transformer 在每个子层的输入和输出之间引入残差连接，使梯度可以顺畅传播并有助于模型的深度训练。如公式2-8所示，

$$ANRAttention = LayerNorm(x + MultiHead(Q, K, V)) \tag{2-8}$$

解码器使用与编码器基本相同的结构。Transformer 的解码是单向实现的，解码器在自注意力机制中使用下三角的 mask 矩阵，该矩阵能把将当前位置以后的信息都屏蔽掉，使前面的解码步骤看不到后面的翻译结果，基于此来确保模型只会利用先前的信息进行计算。并且，Transformer 还在解码器的自注意力模块和前馈神经网络模块之间，设计了一个编码器-解码器注意力模块，其计算如公式2-9所示，该模块可以把解码器的注意力集中到源端相关的词语上，其中，Q 是解码器的输入，K 和 V 是编码器的输出。

$$DecoderLayer(x, EncoderOutput) = LayerNorm(x + MultiHead(Q, K, V)) \tag{2-9}$$

最后，如公式2-10所示，使用 Softmax 函数计算各个位置上词语的概率分布。

$$
\begin{aligned}
Linear(x) &= xW^O + b^O \\
Result &= Softmax(Linear(x))
\end{aligned}
\tag{2-10}
$$

Transformer 的设计极大地推动了深度学习模型在序列处理任务中的发展，成为了目前自然语言处理领域的主流模型之一。它采用了无监督的预训练和有监督的微调两个阶段，在预训练阶段，模型通过自回归任务学习语言模型；在微调阶段，模型通过监督任务，如机器翻译，进行微调。Transformer 在自然语言处理任务中取得了巨大成功，成为了文本生成、机器翻译等任务的标配模型。

2.5 本章小结

本章综述了与大语言模型代码生成的相关的研究与技术，涵盖了信息检索增强和 Transformer 技术相关应用领域和知识。根据现有研究，针对大语言模型的代码生成方法主要集中在引入新的模型结构或者强化推理提示词和信息检索方法两方面。在现有研究的基础上，本文在第三章提出了新的算法模板库方法。

第三章　算法模板库方法

本章描述本文提出的基于算法模板库的大语言模型代码生成方法，简称为ATI（Algorithm Template Inference）。首先结合已有方法调研介绍 ATI 方法提出的动机，然后再概述整体方法，并详细介绍该方法的设计结构。

3.1　研究动机

虽然一系列大型语言模型，例如 GPT-4、PaLM 和 LLaMA 在各种推理任务中都展示了令人瞩目的性能。但关于如何进一步增强大语言模型的推理能力的争论依然激烈，抛开扩大模型参数规模来提高推理能力这一方向外，应用更有效的提示方法也可以进一步增强大语言模型的功能和性能，但是通过对已有研究方法的调研我们发现在该研究方向上也存在有以下问题：

（1）思维链提示的局限性

由于通过扩大模型参数的方式来提高大语言模型的推理能力会产生大量的成本消耗，很少有机构能够承担这些成本。同时，这种方式也不具有通用性。因此，在实际生产过程中，针对具体问题来构建高质量提示链的方式反而更受欢迎，也更具有普适性。目前，研究人员们常从两个方面来构建提示链：(1) 单查询推理：这些方法 [2, 34, 40] 通常侧重于提示工程，其推理过程可以在单个查询中完成，例如 CoT 在输入查询后附加“让我们一步一步思考”以提高推理准确率，或者提供少样本提示让它提供与任务相关的范例来协助答案生成；(2) 多查询推理：这些方法 [9, 41] 侧重于利用多个大语言模型查询来引出不同的合理推理路径，从而将复杂问题分解为一系列更简单的子问题，例如 Least-to-Most[8]、ToT[9] 和 GoT[10]。

但这些方法也具有一定的局限性，对于单查询推理，它通常需要事先假设推理过程的相关样例，这使得手动逐个任务设计它们变得不切实际，因此缺乏针对性；同时单查询推理只能基于一次查询所获得的信息进行推理。如果查询结果不够全面或准确，推理结果可能会受到影响，导致错误的结论。其次，对于需要多步推理或综合多个信息源的问题，单查询推理显得不足，单查询推理

通常难以充分理解复杂问题的上下文，尤其是在涉及多个主题或跨领域的问题时难以得出复杂问题的正确答案。最后，单查询推理高度依赖于初始查询的输入质量。如果输入信息模糊或不完整，推理的准确性会大大降低。多查询推理由于推理路径的递归扩展以及寻找推理过程背后所特有的内在结构，导致其对于每一项具体任务的求解往往是计算密集型的，这就代表其需要较高的查询成本和时间开销，尤其是在处理大量数据或复杂问题时。并且多次查询可能会产生大量的信息，容易导致信息过载，也会让如何从中筛选和整合出有用的信息成为一个挑战，这些都会影响推理质量。不同查询结果之间可能存在不一致或矛盾的情况，随着查询次数的增加，推理过程的复杂性也随之增加，这可能导致模型难以处理在推理过程中出现错误。

最为重要的是，在真实的软件开发人员针对某一个具体问题考虑生成代码的时候，他往往不是从头开始推理的，人类的思维模式很大程度上会受个人经验的影响，这即包括学习经验的影响也包括日常习惯经验的影响，放到代码生成任务上就体现在软件开发人员会先从以往的开发经历中寻找一个与当前需求最相似的代码框架或模版，并在此基础上对现有问题进行抽象并提取关键信息，然后思考这些提取出来的信息能否与已有的经验相结合，最后根据这种经验结合的方法来解决新的问题。所以我们看到，不论是单查询方法还是多查询方法，都没有很好的利用历史的思维和经验，这对于代码生成问题是很不利的。

（2）大语言模型在代码生成上受到质疑

大语言模型的兴起引起了人工智能界的极大兴趣。这些模型，尤其是 GPT-4 已经被广泛应用于各行各业来帮助从业者进行劳动生产活动 [42]。然而，如何准确评估大语言模型的代码生成能力和数据污染的程度却成为了一个悬而未决的新问题。为了解决以上问题，Yiming Huang 等人 [1] 通过考察大语言模型的代码生成能力来评估其推理能力，尤其是针对竞赛级代码题，这些问题由专家精心设计，具有严格的竞赛属性。它们质量高，独一无二，并表现出出色的判别能力，测试用例也经过精心准备，这些问题对人类程序员和大语言模型都提出了重大挑战。这些题目要求大语言模型能够从出题人所呈现的情景推测出解题思路，这就要求大语言模型能够对算法有透彻的理解能力，能够将推理与编码技巧相结合，并且具有较强的解题能力。并且由于竞赛级代码题中严格的问题选择过程，也间接的降低了数据污染的可能性。但不幸的是，研究表明，GPT-4 在 2021 年 9 月之后发布的编程问题上的表现明显更差，这也是

促使人们对其推理能力产生怀疑的最直接原因。研究人员们深入研究了 GPT-4 在不同时间的编程问题上的表现，使用 pass@ k 指标来衡量 GPT-4 的性能。如表 3-1所示。其中，D1 是简单难度的问题，D2 是中等难度的问题，D3 是困难难度的问题。时间 1 是 2010 年 10 月至 2021 年 9 月发布的问题，时间 2 是 2021 年 10 月至 2023 年 11 月发布的问题。

表 3-1: GPT-4 在不同问题组上的表现

Metric	D1		D2		D3	
	Time1	Time2	Time1	Time2	Time1	Time2
pass@1	78.11 %	10.54 %	42.38 %	0.61 %	9.45 %	0.18 %

对于 2021 年 9 月以前的问题，GPT-4 在不同难度的问题中只表现出了轻微的波动。然而，对于 2021 年 9 月之后发布的问题，却产生了与正常波动范围明显的偏差。有趣的是，这个时间与 OpenAI8 宣布的 GPT-4 训练数据截止日期相吻合。这一观察结果点明了大语言模型在没有类似预训练数据的情况下解决未见过的编程问题时所面临的挑战。在模型训练范围之外的问题上观察到的性能下降可能源于推理和泛化的局限性。正如 Yadlowsky [43] 等人所强调的那样，当面对超出其预训练数据的问题时，Transformer 模型会表现出各种故障模式，并且其泛化能力会下降，即使对于简单的问题也是如此。同样，Lu[1] 等人认为，大型语言模型的卓越能力主要源于上下文学习，并不一定反映推理能力的出现。在未见过的问题上观察到的性能下降引发了人们对 GPT-4 内在推理和泛化能力的严重质疑。这表明模型可能过度依赖训练中的模式识别和再现，而不是掌握基本原理并将其应用于新问题。这一观察结果与最近关于大型模型数据记忆倾向的争论相一致（Carlini[44] 等人，2023 年；Yang [45] 等人，2023 年）。因此，探索增强模型泛化能力和减少对预训练数据的依赖的方法开始变得至关重要。

针对错误问题进行分析，Yiming Huang 等人 [1] 发现 GPT-4 在第一个测试用例上就出错的情况占错误总数的 70%，这表明模型在解决问题的初始阶段就经常遇到困难，可能是因为难以理解问题的要求导致无法根据给定的测试用例生成正确的解决方案。尤其是，2021 年至 2023 年期间发布的问题上出现该种错误的比例显著增加。本文认为这表明大语言模型在解决看不见的问题时更有可能面临理解和推理方面的挑战。

3.2 算法模板库方法概述

根据上述研究动机分析，本文提出了一种基于问题解析器和算法模板的大语言模型的代码生成方法，简称为 ATI（Algorithm Template Inference）方法。ATI 方法旨在设计一个新颖且多功能的思维增强推理框架，用来提高大语言模型在代码生成任务中的推理准确性、效率和鲁棒性，其主要流程如图3-1所示。 具体地说，我们从不同的解题过程中提炼出包含一系列算法模板的轻量

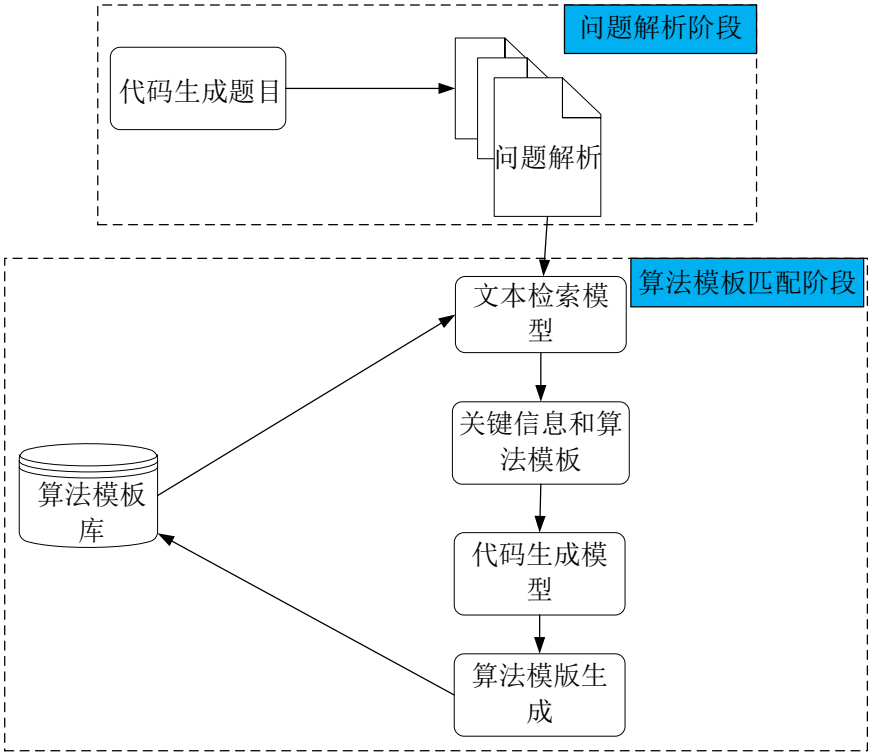

图 3-1: ATI 流程图

库-算法模板库，并可在任务之间共享。然后，对于每个问题，我们先使用问题解析器提取问题的关键信息，这些信息涵盖题目类型，问题变量和约束等，并利用这些信息去检索一个相关的算法模板，使用特定的推理结构对其进行实例化，以实现高效的思维增强推理。为了保证 ATI 的可扩展性和稳定性，我们进一步提出了算法模板库管理器来动态更新算法模板库，随着越来越多的问题被解决，算法模板库也会被进一步丰富。

我们的方法有三个关键优势：

- 准确度提升：利用共享的算法模板，我们可以自适应地实例化高级思维以

解决不同的任务，无需从头开始构建推理结构，从而提高推理准确度。

- 推理效率：我们的算法模板增强推理可以直接利用信息丰富的历史推理结构进行推理，而无需复杂的多查询过程，从而提高推理效率。
- 模型鲁棒性：使大语言模型在解决同类问题时能够始终如一，从而显著增强我们方法的模型鲁棒性，就像人类从思维检索到思维实例化的思维过程一样。我们的实证研究表明，算法模板库显著增加了大语言模型代码生成的准确度、效率和鲁棒性。

在这里，我们总结了我们的研究结果：

1. 我们提出了一种新颖的大语言模型推理增强方法-算法模板库，以提高基于大语言模型的代码生成的准确性、效率和鲁棒性。
2. 我们提出算法模板库来存储从不同问题中提炼出的丰富的算法模板，并自适应地实例化每种算法模板以解决每个特定任务。
3. 我们设计了算法模板库管理器来从各种解决方案中提取算法模板，并随着更多任务的解决而不断提高算法模板库的容量。

3.3 问题解析阶段

本小节对 ATI 方法中的问题解析器的具体实现方法和设计原理进行了详细的介绍。通过对上文中提到的多种思维增强方法的研究我们可以发现，不论是单查询推理还是多查询推理，他们的推理过程都会受到其设计的范例和推理结构的限制，并且它们都忽略了从以前完成的任务中获取指导思想 [46]，但这些指导或思想对于提高解决类似问题的效率和准确性却都很有帮助。基于此，本文构建了问题解析器来解析问题的关键信息并配合算法模板库使大语言模型能够在没有任何参数更新的情况下有效地解决不同的任务。我们通过在推理活动之前添加问题解析器，要求大语言模型 LLM 在最终推理阶段前先完成任务信息的解析和理解工作，最大程度的降低大语言模型单次推理的负担。具体来说，我们设计了一个问题解析器，配合 LLM 来解析并形式化问题信息。提炼的问题信息可以表示为：

$$x_d = LLM(\phi(x)) \tag{3-1}$$

其中 x 是问题描述，x_d 是提取出来的问题关键信息。问题解析器的详细提示如下图3-2所示。

我们使用问题解析器从输入任务中获取关键元素，重点关注解决问题的基

【问题解析器】：
作为问题解析领域的专家，你擅长从用户输入的题目中提取出解决问题所需的关键信息，并根据题目的类型，熟练地将提取到的信息转换为合适的格式。
请从用户输入的查询中，对解决问题所需的关键信息进行分类和提取，需要被解析出来的信息应包括：
1. 关键信息：从用户输入的题目中提取出关键变量的值和类型，将交给相应的专家进行任务解决，确保提供解决问题所需的所有关键信息。
2. 限制：分析题目的目标和相应的限制。
3. 提炼任务：根据1和2对问题进行扩展，总结出一个可以用来直到代码生成的标准问题。结合问题的测试用例代码以及原始问题中的关键变量类型和数值约束，对标准问题中的关键变量进行限制。之后，以从用户输入中解析出来的信息作为输入，以示例的方式解决问题。

图 3-2: 问题解析器示例

本参数和变量以及输入任务的目标及其相应的约束。然后，我们将这些解析出来的的信息重新组织成清晰易懂的格式，以供后续推理阶段使用。具体的推理过程可以建模为有向图 $G = (V, E)$；V 是一组顶点，$E \subseteq V \times V$ 是一组边。G 是有向的，因此有向图的边可以看作是有序顶点对 $E \subseteq V \times V$ 的子集。顶点包含针对当前问题的解决方法。有向边 (t_1, t_2) 表示思想 t_2 是使用 t_1 作为“直接输入”构建的，即通过明确指示大语言模型使用 t_1 来生成 t_2。在实现过程中只需要从顶点 $v_1, \ldots, v_k$ 中添加连接边即可方便的将推理过程中产生的新方法例如一个新的算法模板聚合到推理路径里，进而生成更长的思维链或者在推理过程中回溯到上一步，选择新的提示思想法 v^+ 构建新的推理路径。

3.4 算法模板匹配阶段

在基于问题解析器对问题进行解析并提取关键信息的基础上，我们使用模板匹配来帮助大模型充分利用历史信息和相似任务的推理思想。本小节介绍了算法模板匹配的设计思路和结构，主要包括对问题分类并抽象出高质量的算法模板，利用 BGE-M3[47] 文本嵌入模型进行多向量检索。最后，利用算法模板

管理器来总结问题解决过程并提炼模板来增加算法模板库的容量。人类在解决问题时经常总结和归纳更高层次的指导方针，然后将其应用于相关问题 [46]。受此启发，我们提出了算法模板库，这是一个轻量级库，其中包含一系列用于代码生成的算法模板。与需要特定指令或示例的传统方法 [33, 48, 49] 不同，我们的算法模板可以在解决不同问题时自适应地实例化，从而以卓越的精度和灵活性增强大语言模型。算法模板库的设计思路源于我们对竞赛级算法题的深度分析，随着算法竞赛的不断发展和日渐火热，围绕竞赛级算法题的研究也逐渐深入，关于如何快速的提高软件开发初学者的解题能力的研究也早就被提上了日程。Malik Braik[50] 等人将现有的竞赛级算法题分为了 30 大类，这几乎可以涵盖市面上所有的算法题目，这也证明了由人类设计的竞赛级算法题和竞赛级奥数题类似，都是具有明确的题目分类的，这也为我们构建算法模板提供了理论基础。虽然这种分类方法能够包含最多的竞赛级题目，但不是每种分类出来的题目都具有明确的解题范式，这也就表明并不是所有问题都能找到一个或几个交叉式的范式组合。为了更好的规范和分析常见的竞赛级算法题，我们分析了 LeetCode[①]网站上包括简单、中等和复杂难度在内的三千道算法题，整理出了具有规范性的 22 种题目分类。但同时，当我们让大语言模型生成该类题目的代码时，发现即使对于同一类型的题目，大语言模型的推理水平也是大相径庭，甚至会产生很偏颇的题目理解。本文认为这是大语言模型在题目理解的时候，无法像人类一样专注题目的关键信息，尤其是在题目描述中引入了与解题内容无关的故事描述时这种误读会显得更加严重。同时，题目的关键信息不只是体现在题面描述，题目的时间限制和空间限制也是区分题目类型的关键，但是大语言模型的分析过程在该方面并没有投入太多的关注，这就导致题目的解答过程会与标准答案产生较大的偏颇 [51]，这也对代码生成的正确率产生了较大的影响。综上分析，为了应对大语言模型在代码生成问题上暴露出的题目理解、思维推理等弊端，本文采用了在代码生成提示过程中引入算法模板库的方法。首先我们在 22 种算法类型中提取出作为推理指导的算法模板，这种基于类型的分类可以便于模板检索，以便找到不同问题的最合适的解决方案。图3–3中给出了一种示例的算法模板，对于滑动窗口类型的算法题，我们研究发现，解决该类问题的首要思路就是先找到窗口的左边界和右边界，并进一步确定的在遍历过程中窗口的更新策略。

因此在解决该类题目时，可以先从题目和约束中分析出模板需要的关键信

[①]https://leetcode.cn/

【滑动窗口模板】：
Define two pointers, left and right, to point to the beginning and end of the interval, respectively. Note that this is a interclosed val. Define sums to track the occurrence count of each character within the interval.

The first while loop checks whether the right pointer has moved beyond the array boundary. Every time the right pointer reaches a new position, the sum/count at the right pointer is updated.

The second while loop moves the left pointer to the right until the interval [left, right] meets the problem's criteria. Each time the left pointer moves to a new position, the sum/count at the left pointer is reduced.

After the second while loop, a valid [left, right] interval that meets the criteria has been found. The problem asks for the maximum interval length, so update res with max(res, current interval length).

The right pointer moves one step to the right each time, exploring a new interval.

The sums in the template needs to be modified according to the problem's requirements. In this case, it's a sum problem, so sums is defined as an integer for summation. If it's a counting problem, sums should be changed to a dictionary for counting. Whenever the left or right pointer changes, the sums must be updated.

```python
def findSubArray(nums):
    N = len(nums)
    left, right = 0, 0
    sums = 0
    res = 0
    while right < N:
        sums += nums[right]
        while 区间[left, right]不符合题意:
        sums -= nums[left]
        left += 1
    res = max(res, right - left + 1)
    right += 1
return res
```

图 3-3: 滑动窗口类型算法题模板

息，然后将信息带入模板中来解题，这样的思维方式类似人类思考方法中的公式思维，虽然公式思维并不是一个解决复杂问题的高效方法，但却是解决规范化问题的有效方法。为了验证模板覆盖的广泛程度，我们对 LeetCode 上标有该标签的题目进行了详细的统计，结果表明该基础策略可以涵盖 LeetCode 上 130 道滑动窗口类算法题中的 80 道题左右。基于此，本文提出了以下研究方法，参照 Ling Yang 等人的研究 [46] 我们将算法模板、模板描述及其对应的类别表示为（T_i，D_{T_i}，C_k），其中，i 表示算法的索引，C_k 表示算法类型，D_{T_i} 是算法模板的描述。对于每个任务，我们使用 BGE-M3 计算 D_{T_i} 和 x_d 之间的多向量嵌入相似度来检索与提炼出与问题 x_d 高度相似的思维模板 T_i。检索过程可以表

示为：

$$j = \text{argmax}_i \left(\text{Sim}\left(f(x_d), \{f(D_{T_i})\}_{i=1}^{N}\right)\right), \quad \text{where Sim}\left(f(x_d), \{f(D_{T_i})\}_{i=0}^{n}\right) >= \delta, \tag{3-2}$$

其中 N 是算法模板库的大小，$f(.)$ 是 BGE-M3 文本嵌入模型，T_j 表示检索到的算法模板。我们设置了一个阈值，只有相似度高于这个阈值的模板才会被我们用来进行模板推理，因为在匹配过程中满足阈值相似度的模板可能有多个，所以我们总会选择相似度最高的模板 $\text{argmax}_i \left(\text{Sim}\left(f(x_d), \{f(D_{T_i})\}_{i=1}^{N}\right)\right)$ 用于本次推理。同时，该阈值也用来界定是否应该从该算法题目中提取出一个新的模板。因此，如果 $\text{Sim}\left(f(x_d), \{f(D_{T_i})\}_{i=0}^{n}\right) < \delta$，我们会在代码生成任务完成后为这类题目生成新的模板。

在实例化推理阶段，我们讨论两种实例化推理的情况，具体取决于当前题目是否能找一个相似度够高的算法模板：第一种情况是我们成功地为该题目检索到一个算法模板 T_j，在这种情况下我们的算法模板增强推理将根据我们提取的问题关键信息和算法模板自适应地实例化为合适的推理结构。在第二种情况下，题目被识别为新的题目，我们会从题目中提取算法模板，然后根据模板类型将其存入算法模板库中。

在遇到新的问题时，我们提出算法模板库管理器来总结在新问题过程中获得的高层指导方针和思想。它可以将每个具体的解决方案推广到更多问题，将关键的提炼知识以算法模板的形式存储在算法模板库中。与为每个问题临时生成样本或说明的方法相比，我们的算法模板库管理器可以确保基于大语言模型的代码生成方法的准确性、效率和鲁棒性的永久性提升。针对什么时候该存储一个新的算法模型的问题，如果我们初始化了一个空的算法模板库或者遇到一个没有合适算法模板的问题，那么基于这道题目提取出的算法模版会被直接存放在算法模板库中。对于其他情况，为了避免算法模板库的冗余，同时又能保留不断生成新的推理思维，我们将计算新模板 $D_{T_{\text{new}}}$ 和模板库中现有模板 $(D_{T_i})_{i=0}n$ 的嵌入向量之间的相似度，并按照以下规则更新算法模板库：

$$\text{Max}\left(\text{Sim}\left(f(D_{T_{\text{new}}}), \{f(D_{T_i})\}_{i=0}^{n}\right)\right) < \delta. \tag{3-3}$$

如果新模板与模板库中所有模板的最大相似度 MAX 依然小于阈值，那么就证明新模板具有模板库中现有模板所不具有的知识，需要更新到模板库中。否则，就不需要进行更新。我们的动态更新策略有效地减少了模板检索的计算负

担，同时确保了算法模板库的轻量级特性。

3.5 本章小结

本章主要详细介绍了我们设计的算法模板库方法的模型和设计。对于现有的基于提示链的大语言模型的代码生成方面存在的推理能力不足、未充分利用历史信息和推理经验的现状，本文采用问题解析器和算法模板库融合的方法，在不改变模型规模的情况下提高大语言模型的代码生成的能力。在构建提示词部分，本文设计了问题解析器，来提取问题中的关键信息并指导大模型的推理过程；在利用算法模版库进行推理增强生成阶段，本文的方法使用 BGE-M3 文本嵌入模型，通过计算题目与算法模板的多向量嵌入相似度来选取最接近的算法模板，并设计了模板库管理器在遇到新问题时更新算法模板库。

第四章　实验设计与结果分析

本章介绍 ATI 方法实验的相关设计，包括基准方法，实验数据，评价指标和参数设置，最后分析实验结果并进行消融实验。

4.1　实验设计

实验设计部分包括实验选择的基准方法、实验数据、实验评估指标、实验环境和参数设置等实现细节。

4.1.1　基准方法

综合已有文献研究 [52–54] 和相关开源代码的情况，我们使用 CODE-QWEN-7B 作为 ATI 实验的基础模型，并进行了主要实验和消融实验。为了证明我们的实验可以广泛的提高各种大语言模型的代码生成能力，本文选取了 1 种基于代码预训练来针对性提高代码生成的大语言模型和一种与我们基础模型同源的，但是拥有更大参数规模的模型。由于很多专业的大语言模型都没有提供高质量的开源的代码，为了保证复现已有工作的准确性，本文对开源的相关模型进行调研和挑选，最后选择了 CodeGeeX-4 这一个 2024 年新推出的针对代码生成能力增强的模型和 CODE-QWEN-14B 这一在 CODE-QWEN-7B 基础上进一步扩展自身参数规模的模型。我们将我们的算法模板库方法与以下提示方法进行了比较：1. 标准提示：这是我们最基本的基线，该实验要求大语言模型直接根据输入查询生成响应，而无需任何特定的指导输入输出的示例或超出查询中包含的任务描述的其他说明。2. 单查询方法：这包括零样本 CoT[2, 4] 和 PAL[34]，它们使用大语言模型分析自然语言问题并生成中间推理步骤。我们还引入了专家提示 [40]，它根据查询输入的特定上下文来创建定制的专家身份，然后将与该专家身份相关的配置文件集成到输入中以生成充分的响应。3. 多查询方法：包括 ToT[9] 和 GoT[10]，它们使大语言模型能够通过考虑多种推理路径和自我评估来确定下一步行动，从而做出深思熟虑的决策。这些方法还允许在必要时进行前瞻或回溯以做出全局决策。为了能够全面检测我们算法模板方

法的代码生成能力，避免在某一种程序设计语言上出现“幻觉”，我们选择了多语言的数据集，并且在 Java 和 Python 两种程序设计语言上都进行了代码生成测试。

4.1.2 实验数据

为了评估我们提出的“算法模板库”的有效性并与以前的方法进行比较，我们选取了 HumanEval-X[13] 数据集，它是在 HumanEval[28] 这个被广泛用于代码生成实验的成熟的数据集上开发的多语言版本代码生成数据。对于在 HumanEval-X 中定义的每一个题目，开发人员都为其设计了四种语言（C++、Java、JavaScript 和 Python）来描述题目信息、题目的规范解决方法和题目测试用例。HumanEval-X 中总共拥有 820 个问题解决方案对，具体来说，HumanEval-X 中每个方法都包含以下数据信息：

- **task_id**：用来表示编程语言和数字问题 id，例如，Java/0 代表 Java 中的第 0 个问题；
- **definition**：包括必要库或包的函数声明；
- **docstring**：指定功能和示例输入/输出的描述；
- **prompt**：函数声明结合 docstring；
- **canonical_solution**：手写的示例解答，所有解答都是正确的并可以提交系统运行的；
- **test**：包括测试用例的测试程序。

Mark Chen 等研究人员 [11] 采取了以下措施确保不论针对哪种语言生成的代码都能符合相应语言的编程风格：首先，使用惯用的命名风格，如 Java、Go 和 JavaScript 中的 CamelCase 以及 C++ 中的 snake_case。其次，将 Java、JavaScript、C++ 和 Go 中的文档字符串放在函数声明之前。文档字符串中的符号被修改，例如，在某些语言中，单引号被替换为双引号，关键字如 True/False、None 也被替换。最后，基于特定于语言的行为来细化测试用例，而不是强迫程序对不同的语言返回相同的结果。例如，在将整数转换为二进制字符串时，Python 方法会在字符串前添加前缀“0b”，而 Java 方法 Integer.toBinaryString 则不会，因此我们在 Java 测试用例中删除了该前缀。

在 Qinkai Zheng 等人 [13] 的基础上，为了进一步验证大语言模型对于复杂算法题的生成能力，我们进一步设计了一组代码生成数据 TemplateLeetcode。

我们从 LeetCode 上选取了 400 道涵盖不同数据类型的题目，并且我们在选择题目上，引入了时间基准，我们尽可能的选取了那些提交次数少的，出题时间更新的题目。这一步骤是为了防止大语言模型在这些题目上产生的数据污染。根据 Yiming Huang[1] 等人的研究，大语言模型在面对晚于他模型训练截止时间之后出现的数据具有明显的推理困难。随后，为了更好的训练，验证和评估我们方法的效果，本文根据 HumanEval-X 的数据结构设计，将 LeetCode 题目整理成与其相似的结构。同时，为了进一步考察大语言模型在多语言上的代码生成能力，我们也选择 Python 和 Java 两种程序设计语言来构建数据集，两种语言的题目整理是相互独立的，且都是符合各自语言规范的，并且都能够提交 LeetCode 系统运行。接下来，本文利用 BGE-M3 来对我们整理的算法模板进行多向量嵌入计算，并根据文本向量化的结果进行信息检索，这些检索到的模板会和问题解析器解析后的题目关键信息结合，用于指导大语言模型的代码生成过程。此外，为了提高代码生成的鲁棒性和稳定性，在模板匹配过程中的遇到的相似度低的题目，会提取该问题进行问题解析后的结果一起用来构建新的算法模板，扩充我们的算法模板库。

4.1.3 评估指标和实现细节

一个优秀的评估指标是推动其所在领域发展的基石，在代码生成任务的评估上，主流的方法例如 BLEU[55] 和 CodeBLEU[56] 等都是通过计算 n-gram 来考察生成代码与正确代码之间的字符串相似度。这些方法虽然可以有效的评估大语言模型的代码生成能力，但其依然存在着一些不足，包括：首先，n-gram 准确度没有考虑语法和逻辑的正确性 [57]，导致偏向具有高 n-gram 准确度却有着严重逻辑错误的候选者。其次，n-gram 准确度过于严格，低估了具有相同语义逻辑的不同输出 [58]。第三，n-gram 准确度在通用性和实用性方面较弱，因为它需要针对不同的编程语言以及特定的编译器和所需的计算资源进行设计 [59]，这也导致他难以设计出具有泛用性的标准。最后，虽然代码是由人为设计的，但他和人类的自然语言却有着很大的区别。人类说话写字通常都是从左到右，目前主流的模型通常也会将自然语言以序列的方式处理，比如端到端的神经机器翻译 [60]。而代码则具有天然的树状结构，需要按照其抽象语法树进行编译。因此，如何评估代码的句法结构就变得尤为重要。同时，词义消歧是自然语言处理中的基础研究问题，因为自然语言通常具有歧义和多变的语义。而代码设计要求具有唯一性、规范性和系统性，指令必须唯一且固定。

综上所述，为了更好的评估代码生成的效果，我们需要寻找一种更能符合软件开发人员思维和日常编码实践的评测基准。目前在软件开发领域最令开发人员信服的可以用来评估代码功能正确性的框架就是测试驱动开发框架，它要求任何需求在被发布实施之前都必须将其转换为测试用例，并且能成功通过这些测试用例的程序才会被定义为合格。基于此，几乎所有新代码的集成都要依赖于创建和通过单元测试。本文也选用测试用例来评估生成代码的功能正确性，并使用 pass@ k[18] 测量性能，使其在现实世界中有用。具体来说，我们使用无偏方法来估计 pass@ k[18]：

$$\text{pass@}\ k := \mathbb{E}\left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}\right], n = 200, k \in \{1, 10, 100\} \tag{4-1}$$

其中 n 是总生成数，k 是采样预算（通常 $k \in \{1, 10, 100\}$），c 是通过所有测试用例的样本数。$1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}$ 是单个问题的 pass@ k 估计值，E 是所有问题的 pass@ k 的期望值。在实验中，我们通过对所有测试集问题中的单个问题的 pass@ k 取平均值来得到期望值。

在模板匹配任务上，我们通过在输入题目和算法模板间使用多向量检索[13] 的方法来计算题目和模板的相关度, 作为稠密检索的扩展，多向量方法利用整个输出向量来表示查询和段落：

$$E_q = \text{norm}\left(\mathbf{W}_{mul}^T \mathbf{H_q}\right) E_p = \text{norm}\left(\mathbf{W}_{mul}^T \mathbf{H_p}\right) \tag{4-2}$$

其中 $\mathbf{W}_{mul} \in \mathbb{R}^{d \times d}$ 是可学习的投影矩阵。根据 ColBert[24] 的方法，我们使用后期交互来计算细粒度相关性得分：$s_{mul} \leftarrow \frac{1}{N} \sum_{i=1}^{N} \max_{j=1}^{M} E_q[i] \cdot E_p^T[j]$；N 和 M 分别是查询和段落的长度。

在本文的参数设定中，模板匹配 *s_mul* 设为 0.5。即针对每一次模板匹配，将多向量检索结果大于 0.5 的题目视为成功匹配了算法模板，如果结果小于 0.5，会把该问题视为一个新的题目并从改题目中抽取模板。对于代码生成任务，我们取 k 值为 1。同时我们设置模型的 temperature 为 0.95，max_length 为 1024，top_p 为 0.9。

在实现细节上，本文利用 CodeGeeX 库计算 pass@ k。利用 FlagEmbedding 库计算多向量检索。利用 Huggingface 的 Transformers 库加载和使用预训练模型，并且都是采用官方原文提供的开源预训练权重。

全部实验基于的环境和主要工具版本如表 4-1所示。

表 4-1: 实验环境及工具版本

项目	版本/配置
操作系统	Ubuntu 18.04
GPU	Tesla V100 32GB
CUDA	11.2
Python	3.9.2
PyTorch	2.0
numpy	1.22.0
Transformers	4.40.1

4.2 实验结果分析

4.2.1 实验结果

为了和本文方法所涉及到的研究场景保持一致，本文在基准模型 CODE-QWEN-7B 上使用了上述数据集复现了 5 种传统的大语言模型代码生成方法和本文提出的基于 ATI 的大语言模型代码生成方法。对于已有方法，本文按照原文论述构建提示链，然后对于所有的输出结果，使用 pass@ k 表示模型推理能力的强弱。我们将使用一个具体的题目，来分别介绍我们使用的推理方法和输出结果，为了全面展示实验结果，我们选取了单查询方法中的 CoT 和多查询方法中的 ToT 以及我们设计的 ATI 方法。具体算法题目描述如算法 4.1所示。

算法 4.1 Number of Enclaves

Question:

def numEnclaves(self, grid):"'You are given an $m \times n$ binary matrix grid, where 0 represents a sea cell and 1 represents a land cell.A move consists of walking from one land cell to another adjacent (4-directionally) land cell or walking off the boundary of the grid.Return the number of land cells in grid for which we cannot walk off the boundary of the grid in any number of moves."'

Input1:

grid = [[0,0,0,0],[1,0,1,0],[0,1,1,0],[0,0,0,0]]

Output1:

3

Input2:

grid = [[0,1,1,0],[0,0,1,0],[0,0,1,0],[0,0,0,0]]

Output2:

0

基于 CoT 方法的大语言模型代码生成实验

使用 CoT 方法来指导大语言模型的推理过程如算法 4.2所示：

算法 4.2 CoT:Number of Enclaves

Input:
Let’s think step by step，generate Python code that meets the input and output requirements based on the problem description.
Output:
要解决这个问题，我们需要：
1. 我们首先需要找到所有在边界上的陆地单元格，这些单元格是无法被包围的，因为它们可以直接到达边界。
2. 通过从边界上的所有陆地单元格出发，使用动态规划 DP（Dynamic Programming）搜索整个矩阵，并在搜索过程中统计最大值

```
def numEnclaves(self, grid):
    m, n = len(arr), len(arr[0])
    dp = [[0] * n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            if i == 0 or j == 0:
                dp[i][j] = 1
            else:
                if (i % 2 == 0 and arr[i][j] > arr[i-1][j-1]) or
                    (i % 2 == 1 and arr[i][j] < arr[i-1][j-1]):
                    dp[i][j] = dp[i-1][j-1] + 1
                else:
                    dp[i][j] = 1
            if j > 0:
                dp[i][j] = max(dp[i][j], dp[i][j-1])
    return max(max(row) for row in dp)
```

图 4-1: 使用 CoT 方法的代码实现

大语言模型推理生成的代码如图4-1所示。将第一个测试用例：grid = [[0,1,1,0],[0,0,1,0],[0,0,1,0],[0,0,0,0]] 输入以上代码，我们发现模型最后输出结果是 3，而不是预期结果 0。这就表明在 CODE-QWEN-7B 模型上使用 CoT 构建思维链的方式并不能针对该题目实现正确的结果推理。分析使用 CoT 生成的代码，我们发现 CoT 在代码生成前的一步一步推理阶段就出现了错误，大语言模型在推理过程中错误的把“Number of Enclaves”认为是一个需要使用动态规划方法实现的题目，我们在分析过程中发现这是情有可原的，因为该问题涉及对于复杂矩阵问题的求解，而同时动态规划也是解决该类问题的一个可选的通用

方法，于是大语言模型在 CoT 的过程中就错误地试图将其应用到“Number of Enclaves”问题中，而忽略了问题的实际需求，即标记和统计被包围的陆地单元格。基于此我们可以得出，CoT 一类的单推理方法虽然能在很多时候提高大语言模型的代码推理性能，但其在构建提示链的时候也会给大语言模型产生一些误导，这些提示链中出现的错误往往会逐步扩大并最终影响大语言模型的所有后序推理过程，这也与 Yiming Huang[1] 等人的研究相符合，传统 CoT 方法在面对题目描述复杂和算法过程繁琐的题目上表现不佳，其主要问题就集中在对于第一个测试用例上，很多题目无法通过第一个测试用例就预示着大语言模型从根本上就没有理解这些题目的用意。

这就要求我们不应该只基于单查询的方式去构建提示链，单查询方式虽然简单但其没有采用任何类型的规划、前瞻或回溯来帮助评估其生成的推理过程，这就容易导致如我们算法中显示的“第一个测试用例 [1]”这样的问题发生。这也是所有单推理查询的局限性，只能采用连贯的方式对思想进行采样，无法对已作出的决策进行回溯或自我评估。

基于 ToT 方法的大语言模型代码生成实验

ToT 作为一种多查询推理方法，通过构建一种树形的搜索结构来引导大语言模型进行多次推理，其中包括深度优先（DFS）、广度优先（BFS）以及集束（BEAM）搜索。Shunyu Yao[9] 等人设计的 ToT 推理过程如算法 4.3所示：

算法 4.3 ToT:Number of Enclaves

Require: 题目 Number of Enclaves 的描述 *initial_prompt*, 每个状态下生成的想法数 *num_thoughts*,ToT 搜索的最大步数 *max_steps*, 跟踪的最大状态数 *max_states*, 动态剪枝的阈值 *pruning_threshold*

1: 构建一个使用 *ToT* 推理增强的 *Agent* →”This agent uses the tree_of_thoughts library to generate thoughts.”
2: 基于 *Agent* 构造模型 *model*
3: 使用参数调用 *ToT* 方法返回推理结果 → *solution=tree_of_thoughts.solve*
4: **return** *solution*
5: 算法结束。

ToT 方法在实现时会多次调用大语言模型进行推理，对于每一个问题解决步骤都同时考虑多个潜在可行计划并执行最有希望的计划来扩展现有的计划公式。并利用思维采样和价值反馈有机地整合了规划和决策机制，从而实现了一种遍历决方案树的有效搜索。这都得益于 ToT 内部的自我反思机制，在多次推

理的过程中，ToT 会要求大语言模型根据候选项们提供的反馈来动态的规划下一步的推理方向或者对已有的候选推理进行剪枝和束集，以此来提到大语言模型的代码生成的准确性。但同时我们也发现，使用 Shunyu Yao[9] 等人设计的 ToT 推理过程需要手动的编写代码来实现对于大语言模型的多次调用，但是思维链推理的主要优势之一就是可以只通过对大语言模型进行单句提示即可增强推理效果。Hulbert （2023）[①]利用 ToT 风格的提示来改进 CoT，实现了单句提示即可调用 ToT 推理，具体实现如算法4.4所示：

```
def numEnclaves(self, grid):
    def dfs(i, j):
        if i < 0 or j < 0 or i >= len(grid) or j >=
            len(grid[0]) or grid[i][j] == 0:
            return
        grid[i][j] = 0
        dfs(i+1, j)
        dfs(i-1, j)
        dfs(i, j+1)
        dfs(i, j-1)

    for i in range(len(grid)):
        for j in [0, len(grid[0]) - 1]:
            if grid[i][j] == 1:
                dfs(i, j)
    for i in [0, len(grid) - 1]:
        for j in range(len(grid[0])):
            if grid[i][j] == 1:
                dfs(i, j)

    return sum(sum(row) for row in grid)
```

图 4–2: 使用 ToT 方法的代码实现

大语言模型推理生成的代码如图4–2所示。使用 ToT 方法生成的代码可以通过所有的测试用例，这证明在 CODE-QWEN-7B 模型上使用 ToT 构建思维链的方式可以针对该题目生成正确的结果推理。在大语言模型使用 ToT 进行推理的过程中，我们发现三名专家的推理过程虽然不尽相同，但都是正确的推理思路，最后大语言模型基于这三种推理路径选出了一套投票数最多的方法，并根据其实例化了代码。这套多查询推理流程可以极大程度上的弥补 CoT 方法中出现的“第一个测试用例”问题，如上述推理过程中展示的那样，ToT 模型并没有完全按照任何一个专家的思维来构建代码生成，对于专家一和专家三，ToT

[①]https://github.com/dave1010/tree-of-thought-prompting

算法 4.4 ToT:Number of Enclaves

Input:
Imagine three different experts are answering this question. All experts will write down 1 step of their thinking, then share it with the group. Then all experts will go on to the next step, etc. If any expert realises they're wrong at any point then they leave. The question is:Generate Python code that meets the input and output requirements based on the problem description.

Output:
专家 1 的步骤:
分析问题：专家 1 首先确认要找到被水完全包围的陆地单元格的数量。这些陆地单元格不能通过任何方向到达网格的边界。
开始解决方案：从网格的边界开始遍历，将与边界相连的所有陆地单元格标记为"visited"。
标记内部区域：对于没有标记为"visited" 的内部陆地单元格，计算它们的数量，因为这些就是完全被水包围的陆地。
实现代码：专家 1 开始写代码，使用 DFS 或 BFS。
专家 2 的步骤:
分析问题：专家 2 确认需要计算完全被水包围的陆地单元格的数量。这些陆地单元格不应该能够到达网格的边界。
开始解决方案：从网格的边缘开始遍历，将与边缘相连的所有陆地单元格从候选列表中删除。
查找内部陆地单元格：遍历网格中的所有单元格，如果是陆地且没有被删除（标记），则将其计入最终结果。
实现代码：专家 2 编写代码，使用迭代或递归方式。
专家 3 的步骤:
分析问题：专家 3 确认任务是找到无法到达边界的陆地单元格。关键是识别与边界不相连的陆地。
从边界开始遍历：专家 3 决定从边界开始，使用队列来标记所有可以到达的陆地单元格。
计算结果：遍历整个网格，计算那些未被标记的陆地单元格数量。
实现代码：专家 3 计划使用 BFS。
综合专家意见的代码实现:
在这个过程中没有专家被淘汰，最终代码将在下面展示

综合他们的分析和代码生成过程，选取了 DFS 作为最后的解题方式。对于专家二，尽管他的思考过程是对的，但他的思维逻辑却和专家一以及专家三相差很大，这种情况也解释了为什么单查询过程中会生成很明显错误的答案，大语言模型在推理的时候是随机的且多方面的，其中就包括一些看似合理实则和正确结果大相径庭的思路。ToT 等多查询方法会要求大语言模型主动去考虑多种不同的路径并进行自我反省和评估，真正做到“深思熟虑”，从而可以最大限度的利用大语言模型的推理性能。

多查询方法虽然可以全面的考虑问题，做出能代表全局的选择。但多次查询会带来很大的时间开销和成本开销，这对于大规模数据推理是极为不利的

基于 ATI 方法的大语言模型的代码生成实验

在本文中我们提出了 ATI 方法来解决以上两种查询方法中暴露出来的问题，我们设计的推理过程如算法 4.5所示：

算法 4.5 ATI:Number of Enclaves

Require: 题目 Number of Enclaves 的描述 *initial_prompt*, 模板匹配阈值 θ

1: 使用问题解析器解析题目描述中的关键信息 x_d
2: 使用 BGE-M3 模型进行对关键信息 x_d 和模板库进行模板匹配，$\theta_d \leftarrow \mathrm{argmax}_i\left(\mathrm{Sim}\left(f(x_d), \{f(D_{T_i})\}_{i=1}^{N}\right)\right)$
3: **if** $\theta_d > \theta$ **then**
4: 　基于算法模板 D_{T_i} 和关键信息 x_d 进行推理
5: **else**
6: 　基于关键信息 x_d 进行推理，从推理结果中提取算法模板存入算法模板库
7: **end if**
8: 算法结束。

虽然本文研究的方法和 ToT 一样都属于多查询方法，但因为本方法需要针对问题信息进行算法模板库检索来找到与问题最相关的思维模板，因此，本方法无法和 ToT 一样改写为单查询，我们将在下文中按照问题解析和算法模板生成两阶段来分别介绍我们的实验结果。ATI 方法针对 Number of Enclaves 算法的问题解析过程如算法 4.6所示：

在问题解析阶段，我们设计的问题解析器可以在复杂的题目描述中提取问题的关键信息，包括问题的类型，我们预设了 22 种算法类型供大语言模型选择，把类型判断作为第一步是为了能让大语言模型使用指定的类型去进行问题变量和约束的提取，也能够更好的指导后面的算法匹配过程。之后我们会提取题目的约束，因为在对市面上各类的算法题目的研究和统计过程中我们发现，题目约束是暗示算法类型的一个非常重要的信息，例如时间复杂为 $\log(n)$ 的题目多涉及二分查找法，因此提取题目中的约束对算法匹配阶段有很好的辅助作用。其次，我们会根据提取到的关键信息和任务约束，将题目抽象为高层次的模型表达，抛弃了无用的修饰，可以帮助大语言模型更深层次的理解题目的本质。最后，我们提取了题目的输入和输入，这样可以进一步规范大语言模型结果输出的格式。ATI 方法针对 Number of Enclaves 算法的算法模板匹配过程如算

算法 4.6 ATI:Number of Enclaves

Output:

关键信息：

问题类型: 深度优先搜索 DFS

输入为一个 m × n 的二进制矩阵 grid，其中 0 表示海洋单元格，1 表示陆地单元格。

移动定义为从一个陆地单元格移动到另一个相邻（四个方向）陆地单元格，或走出网格边界。

目标是返回无法通过任何移动到达边界的陆地单元格数量。

约束：

$1 \leq m, n \leq 500$

grid[i][j] 只能是 0 或 1

任务提取：

从给定的二进制矩阵中，找到所有无法到达边界的陆地单元格的数量。确保算法在大规模数据（最大 500x500 矩阵）下的可行性。算法应能够有效地遍历矩阵，并确保每个单元格的访问复杂度是可控的。

Python 转化：

输入参数：

grid: 二维列表，表示 m × n 的二进制矩阵。

答案形式：

返回整数，表示无法到达边界的陆地单元格数量。

法 4.7所示：

算法 4.7 ATI:Number of Enclaves

Input:

你是一位代码生成领域的专家，擅长将以前的解决问题的方法应用于新问题。用户将提供一套问题解析的结果 x_d 和算法模板 D_{T_i}。你的目标是分析问题解析后的结果，并根据算法模板生成一个可以行的代码，你只需提供代码，其他的让编译器处理。

大语言模型推理生成的代码如图4–3所示。使用 ATI 方法生成的代码可以通过所有测试用例，这证明在 CODE-QWEN-7B 模型上使用 ATI 构建思维链的方式可以针对该题目完成正确的结果推理。在大语言模型使用 ATI 进行算法模板匹配时，我们发现在使用问题解析器返回的信息进行文本嵌入查询后，BGE-M3 成功的匹配到了与该问题最接近的 DFS 类型的算法模板，并且成功实例化了正确的代码。对于一些更加灵活的问题，比如多分类的动态规划问题，我们会针对性的对这类题目设计多个模板，因此在模板匹配时我们可以让文本嵌入模型返回多个算法模板结果，这就可以保证我们在代码推理过程中可以运用回溯的方法重新选择模版指导推理，使大语言模型具有了和使用 ToT 方法一样的自我反省机制，这极大的增加了模型的稳定性和鲁棒性。同时针对算法模

```
    def draw(self, grid, x, y):
        grid[x][y] = 0
        if x-1>0       and grid[x-1][y]: self.draw(grid, x-1, y)
        if x+1<self.height and grid[x+1][y]: self.draw(grid, x+1,
            y)
        if y-1>0       and grid[x][y-1]: self.draw(grid, x, y-1)
        if y+1<self.width and grid[x][y+1]: self.draw(grid, x, y+1)

    def numEnclaves(self, grid: List[List[int]]) -> int:
        self.height, self.width = len(grid), len(grid[0])
        for border in range(self.width):
            if grid[0][border]:
                self.draw(grid, 0, border)
            if grid[self.height-1][border]:
                self.draw(grid, self.height-1, border)
        for border in range(1, self.height-1):
            if grid[border][0]:
                self.draw(grid, border, 0)
            if grid[border][self.width-1]:
                self.draw(grid, border, self.width-1)

        return sum([sum([grid[i][j] for j in range(1,
            self.width-1)]) for i in range(1, self.height-1)])
```

图 4-3: 使用 ATI 方法的代码实现

板匹配度过低的情况，我们也会尝试从问题生成的答案中提炼新的模板，进一步扩充我们的模板库。这虽然会带来内存容量的额外开销，但却可以让大语言模型在推理过程中充分利用过去的推理的经验。最后，因为我们的方法是在具体代码生成之前先匹配好了算法模板，这可以帮助大语言模型构建一条高效率而又准确的推理路径，从而避免了大语言模型在代码生成过程中搜索所有可能性，极大的降低了时间开销

结果分析

如表4-2所示，表格中标为红色的数值是针对单个大语言模型中实验结果最好的数值，标为蓝色的数值是实验结果第二好的数值。

我们的 ATI 方法在这两套极具挑战性的基准测试中始终优于所有以前的提示方法。在 Python 代码生成问题上，以 CODE-QWEN-7B 为基准，我们的方法在 TemplateLeetcode 数据集中实现了 34% 的准确度提升；即使是与在代码生成上表现良好的 ToT 相比，我们的准确度也提高了 7%。在 HumanEval-X 数据集上，我们的方法相比不使用提示链 no CoT 方法，获得了 13% 的性能

表 4-2: 将 ATI 在各个模型上与之前各种增强代码生成方法进行比较

数据集	大语言模型	no CoT	CoT	PAL	ToT	GoT	ATI
HumanEval-X	CODE-QWEN-7B	0.72/0.61	0.75/0.63	0.77/0.65	0.76/0.63	0.75/0.63	0.81/ 0.69
	CODE-QWEN-14B	0.76/0.65	0.78/0.68	0.77/0.68	0.81/0.70	0.81/0.71	0.83/0.73
	CodeGeeX-4	0.76/0.66	0.79/0.68	0.79/0.70	0.81/0.72	0.82/0.72	0.84/0.75
TemplateLeetcode	CODE-QWEN-7B	0.49/0.41	0.55/0.47	0.62/0.56	0.62/0.54	0.61/0.52	0.68/0.59
	CODE-QWEN-14B	0.55/0.49	0.61/0.53	0.68/0.59	0.71/0.61	0.72/0.64	0.77/0.67
	CodeGeeX-4	0.57/0.51	0.62/0.55	0.67/0.59	0.73/0.64	0.74/0.65	0.79/0.67

提升, 相较于表现第二优秀的 PAL 和 ToT 方法也提高了 5%。此外，与最常见的 CoT 方法相比，我们也看到了显著的准确度提升：HumanEval-X 数据集上提升了 8%;TemplateLeetcode 数据集上提高了 19%；在 Java 代码生成问题上，以 CODE-QWEN-7B 为基准，我们的方法在 TemplateLeetcode 数据集中实现了 30% 的准确度提升；与 ToT 相比，我们的准确度提高了 8%。在 HumanEval-X 数据集上，我们的方法相比不使用提示链的 no CoT 方法，获得了 13% 的性能提升, 相较于表现第二优秀的 PAL 方法也提高了 6%。此外，与最常见的 CoT 方法相比，我们也看到了显著的准确度提升：HumanEval-X 数据集上提升了 9%;TemplateLeetcode 数据集上提高了 20%。同比观察我们不难发现，针对多语言代码生成问题，我们选用的三个大语言模型在 Pyhton 代码上的表现往往都是要优于在 Java 代码上的表现，这与实际是相符合的，因为现今的大语言模型主要还是在基于 Python 代码实现的数据集上进行的预训练和微调，这也证明了针对大语言模型的预训练和微调依然是影响大语言模型性能和推理表达能力的关键因素。

同时根据图4-4和图4-5中的指标对比来看，在 CODE-QWEN-14B 上，我们的 ATI 针对这两个数据集的 Python 代码生成问题上分别获得了 9% 和 28% 的推理能力增强，以及在 Java 代码生成问题上的 12% 和 26% 的结果提升。在 CodeGeeX-4 上我们的方法也在 Python 代码生成问题上取得了 10% 和 27% 的提高，以及针对 Java 代码生成问题的 13% 和 23% 的推理水平提高。这也符合本文提出的原始设想，即使构建有效的提示链可以帮助大语言模型提高自身的推理能力，但随着模型自身规模和基础推理能力的增强，所有的提示方法的效果都会减弱。但平行比较所有提示方法，我们的 ATI 方法在面对模型规模不断扩大的情况时，其受到性能减弱的影响也是最小的。这可能是因为我们的方法会在大语言模型推理之前引入算法模板，并利用模版来引导模型推理，由于这部

分的工作是独立于具体推理模型的，所以整体推理能力的提升对于具体推理模型的依赖度是更低的，并且实现引入算法模版也在一定程度上统一了不同大语言模型尤其是参数规模小的大语言模型的推理过程，避免小规模的大语言模型容易陷入错误推理路径的问题。

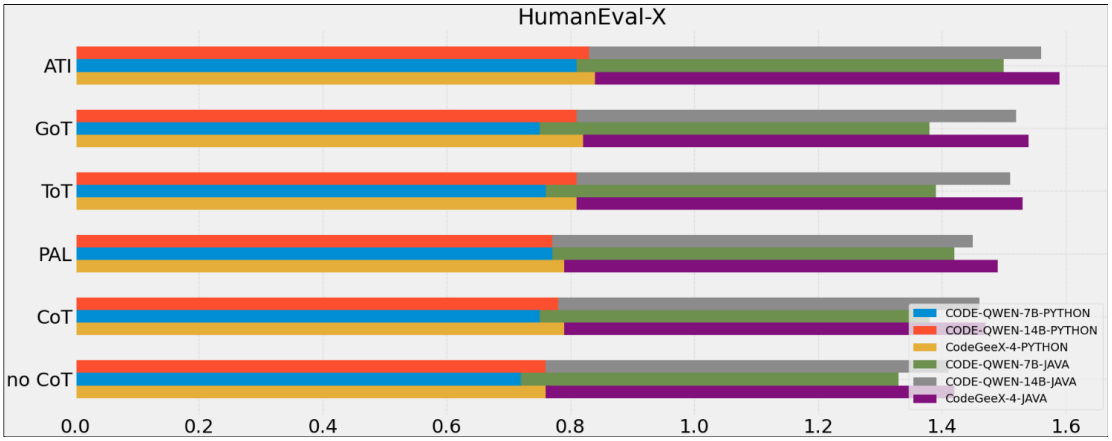

图 4–4: 基于 HumanEvalX 的不同提示链构造方法在三种大语言模型上指标的对比示意图

在进一步的观察中发现，尽管 TemplateLeetcode 数据集中的算法类型与算法模板库更加契合，但在实际运行期间，TemplateLeetcode 数据集的运行效果是要略低于 HumanEval-X。这种现象可能是因为虽然 TemplateLeetcode 数据集在模版匹配时能获得更好的匹配数据，但竞赛级编码题的难度是要明显大于 HumanEval-X 中给出的代码生成例题。同时我们也发现，相比与 CODE-QWEN-7B，CodeGeeX-4 和 CODE-QWEN-14B 在使用相同思维链的情况下，TemplateLeetcode 比起 HumanEval-X 的 pass@ k 下降是更少的。这也说明了大语言模型本身的参数规模依然是限制其代码生成能力的最重要指标。相比之下，基于算法模板的方法虽然也受到大语言模型参数限制的影响，但其下降的幅度却相对较小。这可能是因为基于算法模板的代码生成方法能够充分的利用题目的信息和历史推理逻辑，这可以帮助大语言模型构建一条高效率的推理路径，避免大语言模型因为理解错误 [14] 而无法生成正确代码。尤其是针对那些具有一定复杂度和挑战性的题目，例如竞赛级的代码编程题目，大语言模型在解决问题时可能需要复杂、迭代和启发式的搜索策略来逐案例的解决这些问题，这对大语言模型自身的推理能力和理解能力是一个极大的挑战，但我们的 ATI 利用算法模板中的历史见解和信息指南，能进一步自适应地实例化更优化的推理结构来解决这些复杂问题，并且能把推理的过程和推理思维模板化，迭代式的提高大语言模型的推理能力，从而为充分榨取大语言模型的推理能力提

供帮助。

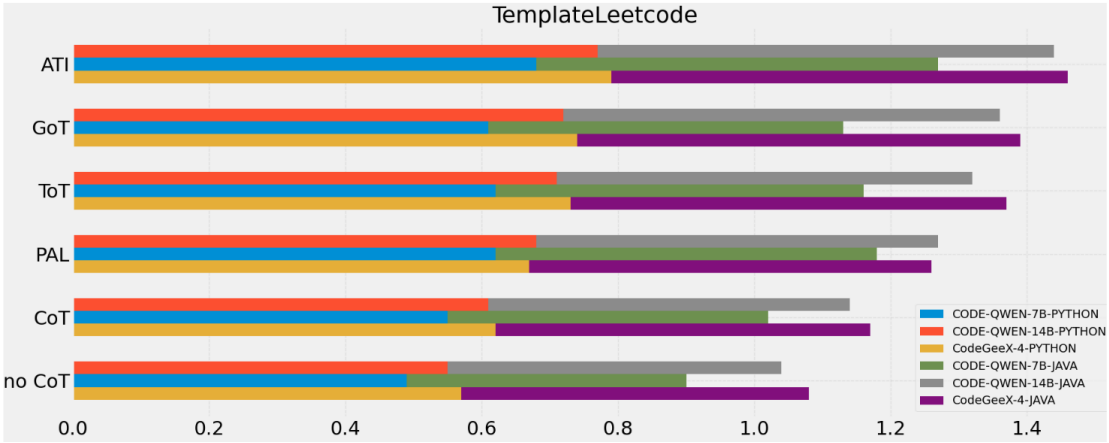

图 4–5: 基于 TemplateLeetcode 的不同提示链构造方法在三种大语言模型上指标的对比示意图

除了准确率显著提升之外，我们的 ATI 方法在推理过程中也表现出来较高的推理效率。纵向对比已有研究方法和本文提出的方法。从实验结果来看，我们的 ATI 虽然在推理时间上略逊于单查询方法，但相比于多查询方法要少得多。通过对结果的进一步分析我们发现，当输入问题可以在算法模板库中找到高匹配率的算法模板时，本文所采用的基于算法模板的推理增强方法不论是在针对 Python 问题的代码生成上还是针对 Java 问题的代码生成上都取得了明显优于其他两种查询方法的结果，除了比基于单查询的方法 CoT 略低外，性能还优于同样是基于单查询方法的 PAL。当输入问题无法在算法模板库中找到符合匹配度要求的模板时，该方法的运行时间会上升，但依然远远优于多查询方法，略低于 PAL 方法，这可能是因为对于匹配度过低的题目，大语言模型需要在生成题目题解的基础上，利用关键信息和题解构建新的算法模板，这也需要一定量的模型计算时间。总体来看，如图4–6所示，我们选取了在运行时间上有代表性的 CoT、PAL 和 ToT 的平均对数运行时间与我们设计的 ATI 方法进行时间对比，在我们的基线模型 CODE-QWEN-7B 上，ATI 方法的平均对数时间比 CoT 高了 17%, 但是比 PAL 方法降低了 6%，与 ToT 方法相比，平均对数时间更是直接降低了 40%。在模型 CODE-QWEN-14B 上，ATI 方法的平均对数时间比 CoT 高了 12%, 但是比 PAL 方法降低了 7%，与 ToT 方法相比，平均对数时间更是直接降低了 38%。在模型 CodeGeeX-4 上，ATI 方法的平均对数时间比 CoT 高了 4%, 但是比 PAL 方法降低了 4%，与 ToT 方法相比，平均对数时间更是直接降低了 44%。

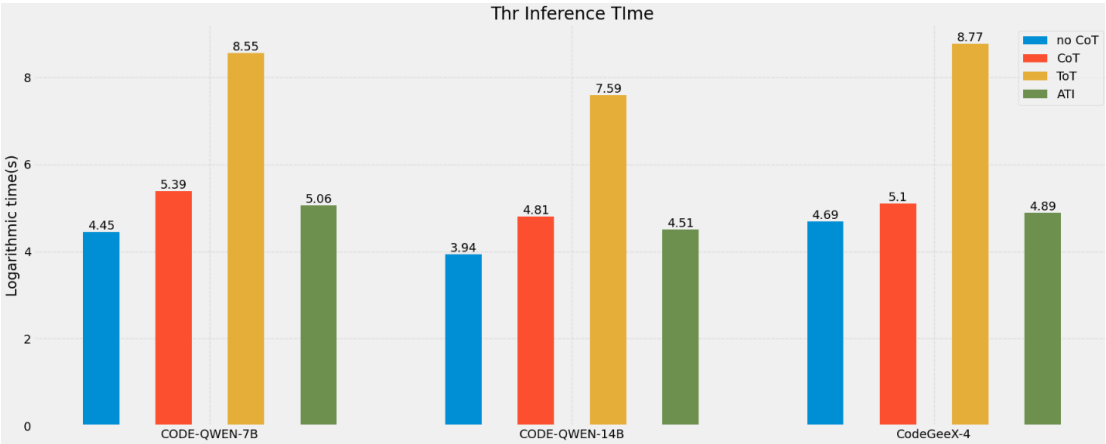

图 4–6: 在不同大语言模型上，我们的 ATI 方法与其他方法的对数推理时间比较

我们研究认为，产生这样数据对比现象的原因是因为单查询方法和多查询方法都需要迭代和启发式搜索来找到可行的解决方案。这个过程特别耗时且效率低下，尤其是对于多查询方法，它还需要进行多条查询搜索以及回溯阶段。相比之下，我们的 ATI 直接以关键信息检索算法模板，可以直接实例化程序来解决问题，从而无需从头开始构建推理结构。尤其是面对满足匹配要求的题目，只需要在调用问题解析器后进行一个查询即可解决问题，大大减少了复杂推理所需的时间。

这表明算法模板库技术在处理未见过的或者具有复杂描述的问题时具有更好的鲁棒性。这些发现为未来大语言模型代码生成的发展提供了重要的见解和指导，提示我们在设计代码生成方法时不仅要考虑方法的准确性，还要充分考虑其在面对庞大且未知的代码库时的适应性和稳定性。

4.3 消融实验

为了分析 ATI 中问题解析部分和算法模版库部分对 ATI 效果的影响，我们设计以下变体实验来比较：

- w/o CoT：该变体在 ATI 中删除了问题解析器的提示链。
- w/o TP：该变体在 ATI 中删除了算法模版库。

实验结果如表 4–3所示：

针对 w/o CoT，当问题解析器被禁用时，所有大语言模型的正确率都经历了一定程度的下降，这说明我们构建的问题解析器确实可以帮助大语言模型更

表 4–3: 消融实验结果

数据集	大语言模型	w/o CoT	w/o TP	ATI
HumanEval-X	CODE-QWEN-7B	0.64	0.60	**0.76**
	CODE-QWEN-14B	0.76	0.71	**0.84**
	CodeGeeX-4	0.78	0.70	**0.85**
TemplateLeetcode	CODE-QWEN-7B	0.54	0.46	**0.68**
	CODE-QWEN-14B	0.63	0.54	**0.77**
	CodeGeeX-4	0.66	0.55	**0.79**

全面更深层次的理解问题，有助于后续的推理。对比三种大语言模型我们可以发现，不论是在 HumanEval-X 数据集上还是在 TemplateLeetcode 数据集上，CODE-QWEN-14B 和 CodeGeeX-4 的正确率下降程度都要小于 CODE-QWEN-7B。这说明随着大语言模型规模的扩张，其对于题目的解析理解能力也会更高，也更容易解析出题目里的关键信息。同时，通过对两个数据集的对比我们发现，对于 TemplateLeetcode 这个更复杂的数据集，大语言模型的准确率下降更明显。这也符合我们上面总结出来的结论，大语言模型更容易地在较简单的任务中提取关键信息，从而使问题解析器的影响不那么明显。相反，在复杂问题中提取关键信息和潜在约束更具挑战性，这使得我们的问题解析器的作用更加突出。

针对 w/o TP，当算法模板库被禁用时，所有大语言模型的推理性能都出现了明显的下降，特别是在需要复杂推理的数据集中。这证明了我们设计的算法模板库在面对代码生成问题时的有效性，也进一步凸显了我们的算法模板库在解决复杂问题方面的优势。

4.3.1 本章小结

本章主要详细介绍了我们实验的具体设计和对实验的进一步分析，通过利用现有的预训练模型应用 ATI 方法进行全面、科学的实验，验证了 ATI 方法的有效性和提高模型性能效率。并设计消融实验验证了 ATI 方法中各结构的必要性和对模型最终性能的影响。

第五章　总结

5.1　工作总结

代码生成能力是考察一个大语言模型推理能力的重要指标。大语言模型的代码生成能力可能因多种因素受到影响，如模型结构的复杂性、数据质量以及训练过程中的问题。优化这些因素不仅能提升模型的推理准确性和效率，还能增强其在实际应用中的实用性。因此，研究如何通过算法模板提升大语言模型的代码生成能力显得尤为关键。

在大语言模型代码生成能力的早期研究中，研究者们常常采用基于统计和规则的方法，将推理任务视作一系列逻辑推导和模式匹配的过程。通过深入分析数据特征和推理路径，这些方法试图提高推理结果的准确性和可解释性。随着深度学习技术的快速发展和成熟，研究方法逐渐转向了基于深度学习模型的应用。利用深度神经网络强大的特征提取和模式识别能力，这些方法通过挖掘输入数据与推理结果之间深层的语义关联，实现了更为精准的推理效果。

具体来说，这些方法通过构建多层神经网络，将输入数据的语义表示在高维隐空间中进行对齐，并结合上下文信息，以此来进行推理。在应用该类方法时，研究人员同时也会结合自注意力机制和图神经网络等技术来增强模型的推理能力。例如，对原有 Chat 模型进行针对性代码生成预训练，这种通过扩大大语言模型参数量级的方式可以显著的提高大语言模型的代码生成能力和推理效率

尽管如此，通过对相关文献的综合调研与实验，我们发现现有方法在实际应用中仍存在一些挑战，如模型推理的时间复杂度较高、推理结果的可解释性不足等。为了克服这些挑战并有效地利用算法模板，我们提出了一种新颖的基于算法模板的大语言模型的代码生成方法，该方法结合了传统的逻辑推理技术和深度学习模型的优势，采用分阶段的推理策略，对输入数据进行多层次的解析和处理。首先，通过问题解析器对数据进行初步分析，提取潜在的推理信息；随后，利用文本嵌入模型对这些数据进行向量表征并基于多向量检索的方式进行模板匹配，以确保推理结果的准确性和可解释性。我们进一步设计了算

法模板库管理器，以不断改进问题解决过程并动态生成算法模板，从而逐步提高大语言模型的推理能力。这种分阶段的方法不仅提高了推理的效率，还增强了模型在复杂推理任务中的表现。

最终，通过一系列实验验证了我们提出方法的有效性。实验结果表明，该方法在多个基准数据集上均取得了显著的性能提升，不仅在推理准确性上有所提高，同时在推理速度和可解释性方面也展现出了良好的性能。综上所述，基于算法模板的大语言模型的代码生成方法为解决当前推理任务中的挑战提供了一种有效的途径，具有广泛的应用前景。

5.2 研究展望

本文认为，尽管我们的方法在保持推理效率和稳健性的同时显著提高了准确性，但在解决需要像人类一样进行创造性思考的问题时，我们方法的增强是有限的，因为这些问题通常不依赖于特定的算法模板。此外，如果我们的 ATI 使用较弱的模型初始化算法模板库，会因为较弱模型的推理能力和指令遵循能力有限，导致派生的算法模版的质量可能不是最优的。但如果通过手动构建算法模板库，又会限制该方法的可扩展性。总的来说，通过算法模板提高大语言模型的代码生成能力的研究前景广阔，未来可以从以下几个方面进行探索和发展：

(1) 设计新的模版特征与融合多种推理方法。虽然目前已有研究尝试将深度学习模型与传统推理方法结合，但这些初步尝试大多较为基础，通常仅限于简单的模型组合。在未来的研究中，我们将进一步探讨如何设计特定的模版特征，并通过融合神经网络与传统算法模板的方法，实现更高效的代码生成能力。这需要大量的实验验证，以确保这些特征在实际代码生成任务中的有效性和鲁棒性。

(2) 探索预训练模型与算法模板的深度结合。本文使用的深度学习模型部分采用了基于 Transformer 架构的预训练模型，这类模型在自然语言处理领域已取得显著成果。然而，在代码生成任务中，针对特定推理需求进行预训练的研究仍相对较少。未来的研究将重点关注如何将预训练模型与算法模板深度结合，开发针对模版推理的预训练方式和模型架构，以提高推理精度和效率。

(3) 拓展代码生成应用的场景与细化粒度。目前的研究多集中于特定场景的推理任务，然而实际应用中，代码生成任务的复杂性和多样性需要更加细化和

广泛的研究。我们计划在未来的研究中，针对不同应用场景和任务需求，细化代码生成的粒度，并结合实时代码生成和多任务学习的方法，提高算法模版的应用广度和深度。例如，在复杂的对话系统中，结合用户历史数据和上下文信息进行精准代码生成，以及在智能助手中，实现实时问题解答和决策支持。

(4) 实验验证与扩展研究。本文提出的基于算法模板的代码生成方法在初步实验中取得了一定效果，未来我们将在更多的代码生成任务和数据集上进行实验，验证该方法的有效性和适用性。同时，将模版推理方法与其他自然语言处理任务结合，探索其在多任务学习中的潜力，以进一步提升模型的综合能力。

总的来说，通过设计新的模版特征、融合多种推理方法、探索预训练模型与算法模板的深度结合，以及拓展应用场景和细化粒度，将为提升大语言模型的代码生成能力提供丰富的研究方向和发展前景。我们期待通过持续的研究和实验，推动这一领域的发展，提升大语言模型在实际应用中的表现。

致　　谢

感谢我的导师葛季栋副教授对我的关心与指导。感谢家人、长辈在我背后的无私付出与包容。感谢同门师兄、师姐和老师对我的帮助。

参考文献

[1] HUANG Y, LIN Z, LIU X, et al. Competition-Level Problems are Effective LLM Evaluators[J/OL], 2024.
https://arxiv.org/abs/2312.02143.

[2] WEI J, WANG X, SCHUURMANS D, et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models[C/OL] // KOYEJO S, MOHAMED S, AGARWAL A, et al. Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022. 2022.
http://papers.nips.cc/paper_files/paper/2022/hash/
9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html.

[3] SHEN S, HOU L, ZHOU Y, et al. Flan-MoE: Scaling Instruction-Finetuned Language Models with Sparse Mixture of Experts[J/OL]. CoRR, 2023, abs/2305.14705.
https://doi.org/10.48550/arXiv.2305.14705.

[4] KOJIMA T, GU S S, REID M, et al. Large Language Models are Zero-Shot Reasoners[C/OL] // KOYEJO S, MOHAMED S, AGARWAL A, et al. Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022. 2022.
http://papers.nips.cc/paper_files/paper/2022/hash/
8bb0d291acd4acf06ef112099c16f326-Abstract-Conference.html.

[5] LI J, LI G, LI Y, et al. Structured Chain-of-Thought Prompting for Code Generation[J/OL], 2023.
https://arxiv.org/abs/2305.06599.

[6] BÖHM C, JACOPINI G. Flow diagrams, turing machines and languages with only two formation rules[J/OL]. Commun. ACM, 1966, 9(5): 366 – 371. https://doi.org/10.1145/355592.365646.

[7] FU Y, PENG H, SABHARWAL A, et al. Complexity-Based Prompting for Multi-step Reasoning[C/OL] // The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023. [S.l.]: OpenReview.net, 2023. https://openreview.net/forum?id=yf1icZHC-l9.

[8] ZHOU D, SCHÄRLI N, HOU L, et al. Least-to-Most Prompting Enables Complex Reasoning in Large Language Models[C/OL] // The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023. [S.l.]: OpenReview.net, 2023. https://openreview.net/forum?id=WZH7099tgfM.

[9] YAO S, YU D, ZHAO J, et al. Tree of Thoughts: Deliberate Problem Solving with Large Language Models[C/OL] // OH A, NAUMANN T, GLOBERSON A, et al. Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023. 2023. http://papers.nips.cc/paper_files/paper/2023/hash/271db9922b8d1f4dd7aaef84ed5ac703-Abstract-Conference.html.

[10] BESTA M, BLACH N, KUBICEK A, et al. Graph of Thoughts: Solving Elaborate Problems with Large Language Models[J/OL]. Proceedings of the AAAI Conference on Artificial Intelligence, 2024, 38(16): 17682 – 17690. http://dx.doi.org/10.1609/aaai.v38i16.29720.

[11] CHEN M, TWOREK J, JUN H, et al. Evaluating Large Language Models Trained on Code[J/OL]. CoRR, 2021, abs/2107.03374. https://arxiv.org/abs/2107.03374.

[12] ROZIèRE B, GEHRING J, GLOECKLE F, et al. Code Llama: Open Foundation Models for Code[J/OL], 2024. https://arxiv.org/abs/2308.12950.

[13] ZHENG Q, XIA X, ZOU X, et al. CodeGeeX: A Pre-Trained Model for Code Generation with Multilingual Benchmarking on HumanEval-X[C/OL] // SINGH A K, SUN Y, AKOGLU L, et al. Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2023, Long Beach, CA, USA, August 6-10, 2023. [S.l.] : ACM, 2023 : 5673 – 5684. https://doi.org/10.1145/3580305.3599790.

[14] LI Y, CHOI D, CHUNG J, et al. Competition-level code generation with AlphaCode[J/OL]. Science, 2022, 378(6624) : 1092 – 1097. http://dx.doi.org/10.1126/science.abq1158.

[15] FRIED D, AGHAJANYAN A, LIN J, et al. InCoder: A Generative Model for Code Infilling and Synthesis[C/OL] // The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023. [S.l.] : OpenReview.net, 2023. https://openreview.net/forum?id=hQwb-lbM6EL.

[16] LI R, ALLAL L B, ZI Y, et al. StarCoder: may the source be with you![J/OL]. Trans. Mach. Learn. Res., 2023, 2023. https://openreview.net/forum?id=KoFOg41haE.

[17] 刘金硕, 文尧. 模板运算代码的自动生成与调优框架 [J/OL]. 计算机工程, 2024, 50(06) : 35 – 47. http://dx.doi.org/10.19678/j.issn.1000-3428.0068234.

[18] KULAL S, PASUPAT P, CHANDRA K, et al. SPoC: Search-based Pseudocode to Code[C/OL] // WALLACH H M, LAROCHELLE H, BEYGELZIMER A, et al. Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada. 2019 : 11883 – 11894. https://proceedings.neurips.cc/paper/2019/hash/7298332f04ac004a0ca44cc69ecf6f6b-Abstract.html.

[19] GAO Y, XIONG Y, GAO X, et al. Retrieval-Augmented Generation for Large Language Models: A Survey[J/OL], 2024. https://arxiv.org/abs/2312.10997.

[20] YASUNAGA M, AGHAJANYAN A, SHI W, et al. Retrieval-Augmented Multimodal Language Modeling[C/OL] // KRAUSE A, BRUNSKILL E, CHO K, et al. Proceedings of Machine Learning Research, Vol 202 : International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA. [S.l.] : PMLR, 2023 : 39755 – 39769. https://proceedings.mlr.press/v202/yasunaga23a.html.

[21] LEWIS P S H, PEREZ E, PIKTUS A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[C/OL] // LAROCHELLE H, RANZATO M, HADSELL R, et al. Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual. 2020. https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html.

[22] GAO L, MA X, LIN J, et al. Precise Zero-Shot Dense Retrieval without Relevance Labels[C/OL] // ROGERS A, BOYD-GRABER J L, OKAZAKI N. Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2023, Toronto, Canada, July 9-14, 2023. [S.l.] : Association for Computational Linguistics, 2023 : 1762 – 1777. https://doi.org/10.18653/v1/2023.acl-long.99.

[23] MUENNIGHOFF N, TAZI N, MAGNE L, et al. MTEB: Massive Text Embedding Benchmark[C/OL] // VLACHOS A, AUGENSTEIN I. Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics, EACL 2023, Dubrovnik, Croatia, May 2-6, 2023. [S.l.] : Association for Computational Linguistics, 2023 : 2006 – 2029. https://doi.org/10.18653/v1/2023.eacl-main.148.

[24] KHATTAB O, ZAHARIA M. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT[C/OL] // HUANG J X, CHANG Y, CHENG X, et al. Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval, SIGIR 2020, Virtual Event, China, July 25-30, 2020. [S.l.] : ACM, 2020 : 39 – 48. https://doi.org/10.1145/3397271.3401075.

[25] ROBERTSON S E, ZARAGOZA H. The Probabilistic Relevance Framework: BM25 and Beyond[J/OL]. Found. Trends Inf. Retr., 2009, 3(4): 333 – 389. https://doi.org/10.1561/1500000019.

[26] KARPUKHIN V, OGUZ B, MIN S, et al. Dense Passage Retrieval for Open-Domain Question Answering[C/OL] // WEBBER B, COHN T, HE Y, et al. Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, EMNLP 2020, Online, November 16-20, 2020. [S.l.]: Association for Computational Linguistics, 2020: 6769 – 6781. https://doi.org/10.18653/v1/2020.emnlp-main.550.

[27] NEELAKANTAN A, XU T, PURI R, et al. Text and Code Embeddings by Contrastive Pre-Training[J/OL], 2022. https://arxiv.org/abs/2201.10005.

[28] DEVLIN J, CHANG M, LEE K, et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding[C/OL] // BURSTEIN J, DORAN C, SOLORIO T. Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers). [S.l.]: Association for Computational Linguistics, 2019: 4171 – 4186. https://doi.org/10.18653/v1/n19-1423.

[29] WU H, QU Y, LIN S, et al. Contrastive Learning for Compact Single Image Dehazing[C/OL] // IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2021, virtual, June 19-25, 2021. [S.l.]: Computer Vision Foundation / IEEE, 2021: 10551 – 10560. https://openaccess.thecvf.com/content/CVPR2021/html/Wu_Contrastive_Learning_for_Compact_Single_Image_Dehazing_CVPR_2021_paper.html.

[30] ABUZAID F, SETHI G, BAILIS P, et al. To Index or Not to Index: Optimizing Exact Maximum Inner Product Search[C/OL] // 35th IEEE International Conference on Data Engineering, ICDE 2019, Macao, China, April 8-11, 2019. [S.l.]:

IEEE, 2019 : 1250 – 1261.
https://doi.org/10.1109/ICDE.2019.00114.

[31] JOHNSON J, DOUZE M, JÉGOU H. Billion-Scale Similarity Search with GPUs[J/OL]. IEEE Trans. Big Data, 2021, 7(3) : 535 – 547.
https://doi.org/10.1109/TBDATA.2019.2921572.

[32] 吴佩泽, 李光辉, 吴津宇. 基于大语言模型的自动化漏洞验证代码生成方法研究 [J], 2024 : 248 – 253.

[33] WANG X, WEI J, SCHUURMANS D, et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models[C/OL] // The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023. [S.l.] : OpenReview.net, 2023.
https://openreview.net/forum?id=1PL1NIMMrw.

[34] GAO L, MADAAN A, ZHOU S, et al. PAL: Program-aided Language Models[C/OL] // KRAUSE A, BRUNSKILL E, CHO K, et al. Proceedings of Machine Learning Research, Vol 202 : International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA. [S.l.] : PMLR, 2023 : 10764 – 10799.
https://proceedings.mlr.press/v202/gao23f.html.

[35] BROWN T B, MANN B, RYDER N, et al. Language Models are Few-Shot Learners[C/OL] // LAROCHELLE H, RANZATO M, HADSELL R, et al. Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual. 2020.
https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html.

[36] GEHRMANN S, ADEWUMI T, AGGARWAL K, et al. The GEM Benchmark: Natural Language Generation, its Evaluation and Metrics[J/OL], 2021.
https://arxiv.org/abs/2102.01672.

[37] ICHTER B, BROHAN A, CHEBOTAR Y, et al. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances[C/OL] // LIU K, KULIC D, ICHNOWSKI J. Proceedings of Machine Learning Research, Vol 205 : Conference on Robot Learning, CoRL 2022, 14-18 December 2022, Auckland, New Zealand. [S.l.] : PMLR, 2022 : 287 – 318.
https://proceedings.mlr.press/v205/ichter23a.html.

[38] CRESWELL A, SHANAHAN M. Faithful Reasoning Using Large Language Models[J/OL], 2022.
https://arxiv.org/abs/2208.14271.

[39] ELGHOUAT A, ALGOUTI A, ALGOUTI A, et al. Attention is all you need for an improved CNN-based flash flood susceptibility modeling. The case of the ungauged Rheraya watershed, Morocco[J/OL], 2024.
https://arxiv.org/abs/2408.02692.

[40] XU B, YANG A, LIN J, et al. ExpertPrompting: Instructing Large Language Models to be Distinguished Experts[J/OL], 2023.
https://arxiv.org/abs/2305.14688.

[41] SUZGUN M, KALAI A T. Meta-Prompting: Enhancing Language Models with Task-Agnostic Scaffolding[J/OL], 2024.
https://arxiv.org/abs/2401.12954.

[42] BUBECK S, CHANDRASEKARAN V, ELDAN R, et al. Sparks of Artificial General Intelligence: Early experiments with GPT-4[J/OL], 2023.
https://arxiv.org/abs/2303.12712.

[43] YADLOWSKY S, DOSHI L, TRIPURANENI N. Pretraining Data Mixtures Enable Narrow Model Selection Capabilities in Transformer Models[J/OL]. CoRR, 2023, abs/2311.00871.
https://doi.org/10.48550/arXiv.2311.00871.

[44] CIBOROWSKA A, DAMEVSKI K. Fast Changeset-based Bug Localization with BERT[J/OL]. Proceedings - International Conference on Software Engineering,

2022, 2022-May : 946 – 957.
http://dx.doi.org/10.1145/3510003.3510042.

[45] YANG S, CHIANG W, ZHENG L, et al. Rethinking Benchmark and Contamination for Language Models with Rephrased Samples[J/OL]. CoRR, 2023, abs/2311.04850.
https://doi.org/10.48550/arXiv.2311.04850.

[46] YANG L, YU Z, ZHANG T, et al. Buffer of Thoughts: Thought-Augmented Reasoning with Large Language Models[J/OL]. CoRR, 2024, abs/2406.04271.
https://doi.org/10.48550/arXiv.2406.04271.

[47] CHEN J, XIAO S, ZHANG P, et al. BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation[J/OL], 2024.
https://arxiv.org/abs/2402.03216.

[48] ZHANG Z, ZHANG A, LI M, et al. Automatic Chain of Thought Prompting in Large Language Models[C/OL] // The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023. [S.l.] : OpenReview.net, 2023.
https://openreview.net/forum?id=5NTt8GFjUHkr.

[49] ZHENG H S, MISHRA S, CHEN X, et al. Take a Step Back: Evoking Reasoning via Abstraction in Large Language Models[C/OL] // The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024. [S.l.] : OpenReview.net, 2024.
https://openreview.net/forum?id=3bq3jsvcQ1.

[50] BRAIK M, HAMMOURI A I, ALZOUBI H, et al. Feature Selection based nature inspired Capuchin Search Algorithm for solving classification problems[J/OL]. Expert Syst. Appl., 2024, 235 : 121128.
https://doi.org/10.1016/j.eswa.2023.121128.

[51] 邹佰翰, 汪莹, 彭鑫, et al. 重新审视代码补全中的检索增强策略 [J/OL]. 软件学报, : 1 – 28.
http://dx.doi.org/10.13328/j.cnki.jos.007226.

[52] SHI W, MIN S, YASUNAGA M, et al. REPLUG: Retrieval-Augmented Black-Box Language Models[J/OL], 2023.
https://arxiv.org/abs/2301.12652.

[53] PRESS O, ZHANG M, MIN S, et al. Measuring and Narrowing the Compositionality Gap in Language Models[C/OL] // BOUAMOR H, PINO J, BALI K. Findings of the Association for Computational Linguistics: EMNLP 2023, Singapore, December 6-10, 2023. [S.l.] : Association for Computational Linguistics, 2023 : 5687 – 5711.
https://doi.org/10.18653/v1/2023.findings-emnlp.378.

[54] KHOT T, TRIVEDI H, FINLAYSON M, et al. Decomposed Prompting: A Modular Approach for Solving Complex Tasks[C/OL] // The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023. [S.l.] : OpenReview.net, 2023.
https://openreview.net/forum?id=_nGgzQjzaRy.

[55] MATHUR N, BALDWIN T, COHN T. Tangled up in BLEU: Reevaluating the Evaluation of Automatic Machine Translation Evaluation Metrics[C/OL] // JURAFSKY D, CHAI J, SCHLUTER N, et al. Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL 2020, Online, July 5-10, 2020. [S.l.] : Association for Computational Linguistics, 2020 : 4984 – 4997.
https://doi.org/10.18653/v1/2020.acl-main.448.

[56] REN S, GUO D, LU S, et al. CodeBLEU: a Method for Automatic Evaluation of Code Synthesis[J/OL], 2020.
https://arxiv.org/abs/2009.10297.

[57] FENG Z, GUO D, TANG D, et al. CodeBERT: A Pre-Trained Model for Programming and Natural Languages[C/OL] // COHN T, HE Y, LIU Y. Findings of

ACL, Vol EMNLP 2020 : Findings of the Association for Computational Linguistics: EMNLP 2020, Online Event, 16-20 November 2020. [S.l.] : Association for Computational Linguistics, 2020 : 1536 – 1547.
https://doi.org/10.18653/v1/2020.findings-emnlp.139.

[58] ALON U, ZILBERSTEIN M, LEVY O, et al. code2vec: learning distributed representations of code[J/OL]. Proc. ACM Program. Lang., 2019, 3(POPL) : 40:1 – 40:29.
https://doi.org/10.1145/3290353.

[59] BARONE A V M, SENNRICH R. A Parallel Corpus of Python Functions and Documentation Strings for Automated Code Documentation and Code Generation[C/OL] // KONDRAK G, WATANABE T. Proceedings of the Eighth International Joint Conference on Natural Language Processing, IJCNLP 2017, Taipei, Taiwan, November 27 - December 1, 2017, Volume 2: Short Papers. [S.l.] : Asian Federation of Natural Language Processing, 2017 : 314 – 319.
https://aclanthology.org/I17-2053/.

[60] BAHDANAU D, CHO K, BENGIO Y. Neural Machine Translation by Jointly Learning to Align and Translate[C/OL] // BENGIO Y, LECUN Y. 3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings. 2015.
http://arxiv.org/abs/1409.0473.

简历与科研成果

基本信息

张浩天，男，汉族，1999 年 6 月出生，河北省沧州市人。

教育背景

2022 年 9 月 — 2024 年 6 月 南京大学软件工程专业 硕士

2017 年 9 月 — 2021 年 6 月 河海大学大学计算机科学与技术 本科

《学位论文出版授权书》

本人完全同意《中国优秀博硕士学位论文全文数据库出版章程》（以下简称“章程”），愿意将本人的学位论文提交“中国学术期刊（光盘版）电子杂志社”在《中国博士学位论文全文数据库》、《中国优秀硕士学位论文全文数据库》中全文发表。《中国博士学位论文全文数据库》、《中国优秀硕士学位论文全文数据库》可以以电子、网络及其他数字媒体形式公开出版，并同意编入《中国知识资源总库》，在《中国博硕士学位论文评价数据库》中使用和在互联网上传播，同意按“章程”规定享受相关权益。

作者签名：_____

_____年_____月_____日

论文题名	基于算法模板推理的大语言模型的代码生成技术研究				
研究生学号	522022320200	所在院系	软件学院	学位年度	XXXX
论文级别	☐ 硕士 ☐ 硕士专业学位 ☐ 博士 ☐ 博士专业学位 （请在方框内画勾）				
作者 Email	522022320200@smail.nju.edu.cn				
导师姓名	葛季栋 副教授				

论文涉密情况：

☐ 不保密

☐ 保密，保密期 (_____年_____月_____日 至 _____年_____月_____日)

注：请将该授权书填写后装订在学位论文最后一页（南大封面）。